UNIVERSIDAD POLITECNICA DE VICTORIA

INVESTIGACIÓN DE LA PRÁCTICA DEL CICLO HAMILTONIANO

Fundamentos de Inteligencia Artificial

PRESENTA

JOSE EMILIANO PUGA SAUCEDO, OSVALDO FABIAN RAMIRO BALBOA, BARBIE VIRIDIANA LUMBRERAS OLVERA, JESUS JUAREZ RAMIREZ y ISRAEL ELISEO CISNEROS SALAS

PROFESOR

Dr. Said Polanco Martagon

Fecha: Noviembre 2026

Índice general

1. Introducción y alcance **4**

2. Teoría de grafos y recocido simulado **5**
2.1. Fundamentos y definiciones formales 5
2.2. Verificación de una ruta 6
2.3. Fundamentos del recocido simulado 6
2.4. Representación, costo y vecindario propuestos 7
2.5. Ejemplo didáctico y parámetros 7
2.6. Pseudocódigo de la adaptación 8

3. Representación del espacio de búsqueda y ACO **9**
3.1. Términos básicos 9
3.2. Tema general: representación del espacio de búsqueda 10
3.2.1. Cómo guardar el grafo 10
3.2.2. Cómo representar una ruta 11
3.2.3. Por qué una permutación válida puede no ser un ciclo . 12
3.2.4. Cómo tratar rutas que no cumplen las reglas 12
3.3. Algoritmo: Optimización por Colonia de Hormigas (ACO) . . 13
3.3.1. Idea 13
3.3.2. Construcción de rutas sin repetir vértices 13
3.3.3. Feromonas: refuerzo y evaporación 13
3.3.4. Información del grafo para elegir el siguiente paso . . . 14
3.3.5. Callejones sin salida y ciclos que no cierran 14
3.4. Ejemplo didáctico con siete vértices 15
3.5. Pseudocódigo y ajustes principales 18
3.6. Aplicación al ciclo hamiltoniano: opciones, dificultades y dudas 19
3.6.1. Opciones de diseño 19
3.6.2. Dificultades previstas 20

3.6.3. Dudas pendientes 20

4. Módulo evolutivo, heurísticas y colonia de abejas 21
4.1. El problema que se quiere resolver 21
4.2. Módulo evolutivo y heurísticas 22
4.2.1. Qué es una heurística 22
4.2.2. Cómo representar y modificar una posible solución . . 22
4.2.3. Función de evaluación, fitness y validez 23
4.2.4. Algoritmo genético (GA) como referencia 24
4.3. Colonia artificial de abejas: versión elegida 26
4.3.1. Origen y alcance de la adaptación 26
4.3.2. Qué ocurre en un ciclo de ABC 26
4.3.3. Selección y abandono 27
4.3.4. Pseudocódigo de la adaptación propuesta 28
4.4. Ejemplo didáctico de seis vértices 29
4.4.1. Qué enseña el ejemplo 31
4.5. Aplicación al ciclo hamiltoniano: opciones y dificultades 31
4.5.1. Criterios para interpretar un resultado 32

5. Métricas de evaluación y evolución diferencial 33
5.1. El problema que se quiere resolver 33
5.2. Métricas de evaluación 34
5.2.1. Qué es una métrica y qué se necesita para medir 34
5.2.2. Tiempo, costo computacional, iteraciones y evaluaciones 34
5.2.3. Tasa de éxito y ejecuciones fallidas 35
5.2.4. Precisión, recall y F1 37
5.2.5. Por qué no basta con un porcentaje de conexiones válidas 37
5.2.6. Qué medidas parecen aplicables 38
5.3. Evolución diferencial: el algoritmo original 39
5.3.1. Origen e idea central 39
5.3.2. Qué ocurre en una generación 39
5.3.3. Parámetros que influyen 40
5.3.4. Cuándo termina la búsqueda 41
5.4. Adaptación a órdenes de visita 41
5.4.1. Por qué no se puede aplicar directamente 41
5.4.2. Función de evaluación y validez 42
5.4.3. Pseudocódigo de la adaptación 42
5.5. Ejemplo didáctico de seis vértices 43

5.5.1. Qué enseña el ejemplo 45
5.6. Aplicación al ciclo hamiltoniano: opciones y dificultades 45
5.6.1. Criterios para interpretar un resultado 46
5.7. Dudas para el profesor 47

6. FHCP Challenge y enjambre de partículas 48
6.1. Métricas del FHCP Challenge 48
6.1.1. Características del benchmark 48
6.1.2. Formato de los archivos 48
6.1.3. Criterios de comparación 49
6.1.4. Reglas históricas y alcance de la comparación 49
6.1.5. Ejemplo completo de entrada y salida 50
6.2. Particle Swarm Optimization (PSO) 51
6.2.1. Alcance de las ecuaciones 51
6.2.2. Decodificación y pseudocódigo propuestos 52
6.2.3. Ejemplo didáctico y dificultades 52

7. Síntesis de diseño para la fase 1 54
7.1. Propuesta de diseño experimental 54
7.2. Precisiones de la integración 55
7.3. Precisión, recall y F1: límites de aplicación 55
7.4. Registro y reproducibilidad 56
7.5. Decisiones pendientes 56

Bibliografía 58

Capítulo 1

Introducción y alcance

El problema del ciclo hamiltoniano consiste en determinar si un grafo contiene un ciclo que visite todos sus vértices exactamente una vez antes de regresar al inicio. Su dificultad combinatoria motiva el estudio de métodos de búsqueda heurística. Esta investigación corresponde a la fase previa a la implementación: fundamenta la representación, la evaluación y las decisiones de diseño de SA, ACO, colonia de abejas, DE y PSO. No presenta resultados experimentales.

El documento integra las investigaciones de las ramas del equipo distintas de main y las complementa con el archivo local investigacion.tex. Se conservan los ejemplos y los pseudocódigos de las aportaciones; los parámetros y adaptaciones se identifican como propuestas, no como resultados comprobados sobre FHCP. Bee Colony se concreta en la variante Artificial Bee Colony (ABC), para mantener una descripción inequívoca de sus fases.

El texto se organiza en cinco bloques: grafos y SA; representación y ACO; heurísticas y ABC; métricas y DE; y FHCP y PSO. La síntesis final relaciona estos bloques con un protocolo de comparación. Las referencias heredadas conservan las notas sobre el alcance de su consulta; una referencia que el autor original declaró pendiente no se presenta como verificada íntegramente en esta integración.

Capítulo 2

Teoría de grafos y recocido simulado

2.1. Fundamentos y definiciones formales

Un grafo simple no dirigido es un par $G = (V, E)$, donde V es un conjunto finito de vértices y E es un conjunto de pares no ordenados de vértices distintos. No hay lazos ni aristas múltiples. Se trabajará con $n = |V| \geq 3$ y $m = |E|$. Una arista representa una conexión, no necesariamente una distancia física. Un recorrido es una secuencia $(v_0, \ldots, v_k)$ tal que $\{v_{i-1}, v_i\} \in E$ para $1 \leq i \leq k$; su longitud es k, el número de aristas recorridas. Un camino simple es un recorrido cuyos vértices son distintos. Un ciclo simple es una secuencia $(v_0, \ldots, v_{k-1}, v_0)$, con $k \geq 3$, cuyos primeros k vértices son distintos y todas las conexiones existen. Se explicita "simple" porque las convenciones terminológicas varían entre textos.[40, 41]

Un camino hamiltoniano contiene todos los vértices exactamente una vez y tiene $n - 1$ aristas. Un ciclo hamiltoniano contiene todos los vértices y tiene longitud exactamente n. Formalmente, para una permutación $\pi = (v_1, \ldots, v_n)$ de V, se requiere

$$\{v_i, v_{i+1}\} \in E \quad (1 \leq i < n), \qquad \{v_n, v_1\} \in E.$$

El primer vértice puede escribirse nuevamente al final para indicar el cierre; no es una segunda visita intermedia. No es necesario recorrer todas las aristas del grafo. El TSP añade costos y busca minimizar el costo del recorrido; aquí se busca existencia, por lo que todos los ciclos válidos tienen la misma longitud combinatoria n.[41, 42]

2.2. Verificación de una ruta

Para comprobar una candidata se verifica que contiene exactamente n identificadores válidos, que no se repite ninguno, que aparecen todos los vértices y que existen las n conexiones, incluida la última hacia el origen. Con consultas de adyacencia de costo constante, esta verificación cuesta $O(n)$ y requiere $O(n)$ espacio auxiliar si se usa una marca por vértice. La complejidad de encontrar una solución no debe confundirse con la de comprobarla.

Considérese $V = \{A, B, C, D, E\}$ y $E = \{AB, BC, CD, DE, EA\}$. La ruta A, B, C, D, E, A es un ciclo de cinco aristas. La permutación A, B, D, C, E contiene todos los vértices, pero falla en BD y CE. Si se elimina EA del grafo, A, B, C, D, E sigue siendo un camino hamiltoniano, pero deja de poder cerrarse. Estos ejemplos proceden de la aportación de grafos y SA; son didácticos.

2.3. Fundamentos del recocido simulado

SA se inspira en el enfriamiento de materiales. Mantiene un estado actual, propone un vecino y compara sus costos. La temperatura $T > 0$ controla la aceptación de empeoramientos; no representa grados físicos en el programa. Para minimizar b, se define $\Delta = b(\pi') - b(\pi)$ y se acepta la candidata con probabilidad

$$P(\text{aceptar}) = \begin{cases} 1, & \Delta \leq 0, \\ \exp(-\Delta/T), & \Delta > 0. \end{cases}$$

La comparación se realiza con el estado actual y se conserva por separado el mejor observado. Aceptar empeoramientos puede permitir escapar de mínimos locales. Al bajar T, un mismo empeoramiento se acepta con menor probabilidad; las mejoras siguen aceptándose.[43, 46]

Hajek estudia condiciones de convergencia con calendarios de enfriamiento específicos. Esas garantías asintóticas no implican que una ejecución finita con enfriamiento geométrico encuentre siempre un ciclo.[44]

2.4. Representación, costo y vecindario propuestos

Se representa cada estado como una permutación de V, con el regreso implícito. Fijar un vértice al comienzo elimina duplicados por rotación sin perder ciclos. Se propone minimizar

$$b(\pi) = \sum_{i=1}^{n} \mathbf{1}[\{v_i, v_{i+1}\} \notin E], \qquad v_{n+1} = v_1.$$

La función cuenta conexiones ausentes: no añade aristas al grafo. Sobre permutaciones, $b = 0$ equivale a un ciclo hamiltoniano. Un valor positivo mide incumplimiento y no certifica una solución. No conviene usar la longitud como segundo objetivo: una permutación circular ya tiene exactamente n conexiones candidatas.

Los movimientos propuestos son intercambiar dos posiciones o invertir un segmento, sin mover el inicio fijo. Ambos conservan los vértices, pero pueden romper conexiones. En grafos dirigidos, invertir un segmento también cambia el sentido de sus aristas internas; esta adaptación se limita a grafos no dirigidos. Restringir la búsqueda a ciclos válidos exigiría disponer de uno inicialmente, lo cual ya resolvería el problema de existencia.

2.5. Ejemplo didáctico y parámetros

En el grafo de cinco vértices anterior se parte de A, B, D, C, E, con $b = 2$. Se elige $T_0 = 2$, un factor de enfriamiento $\alpha = 0.8$ y un intento por temperatura, únicamente para ilustrar el método.

1. Intercambiar B y D produce A, D, B, C, E, con $b = 3$: faltan AD, DB y CE. Como $\Delta = 1$, la aceptación tiene probabilidad $e^{-1/2} \approx 0.607$. Un valor ilustrativo $u = 0.40$ produce aceptación. El mejor costo guardado sigue siendo 2 y la temperatura baja a 1.6.

2. Invertir D, B, C produce A, C, B, D, E, con $b = 2$. Faltan AC y BD; se acepta la mejora y la temperatura baja a 1.28.

3. Intercambiar C y B produce A, B, C, D, E, con $b = 0$. Las aristas AB, BC, CD, DE, EA existen, por lo que se verifica el ciclo y termina la búsqueda.

Los movimientos y el azar se escogieron a mano; no constituyen una medición ni indican que SA deba resolver FHCP en tres pasos.

2.6. Pseudocódigo de la adaptación

1. Recibir G, temperaturas $T_0 > T_{\text{mín}} > 0$, $0 < \alpha < 1$, intentos por nivel $L \geq 1$ y presupuesto total $B \geq 1$.

2. Crear una permutación inicial, evaluar su costo y conservarla como actual y mejor. Verificarla y terminar si ya es un ciclo. Contar también esta evaluación inicial.

3. Mientras queden recursos y $T > T_{\text{mín}}$, realizar hasta L propuestas: elegir dos posiciones distintas y aplicar intercambio o inversión con probabilidad 1/2.

4. Antes de evaluar cada propuesta, comprobar el presupuesto. Evaluarla y aumentar el contador, tanto si se acepta como si se rechaza.

5. Aceptar si $\Delta \leq 0$; en otro caso, generar u uniforme en $[0, 1)$ y aceptar si $u < e^{-\Delta/T}$.

6. Actualizar el mejor estado y comprobar de forma independiente cualquier candidata con costo cero. Si pasa la verificación, devolverla.

7. Tras cada nivel, actualizar $T \leftarrow \alpha T$. Si no se ha detenido, repetir.

8. Al terminar sin ciclo, devolver la mejor candidata y el estado "no encontrado dentro del presupuesto".

El esquema adapta las operaciones descritas en la literatura de SA.[43, 45] Temperatura, calendario, vecindario y escala del costo deben ajustarse conjuntamente. Un enfriamiento rápido reduce exploración; uno lento exige más recursos. Las mesetas y los mínimos locales siguen siendo posibles. Un fallo no demuestra que el grafo sea no hamiltoniano.

Capítulo 3

Representación del espacio de búsqueda y ACO

3.1. Términos básicos

Definición (Grafo). Un grafo no dirigido es un par $G = (V, E)$, donde V es un conjunto de n vértices y E es un conjunto de m aristas; cada arista $\{u, v\}$ une dos vértices distintos. Dos vértices unidos por una arista son vecinos; el grado de un vértice es su número de vecinos [6].

Definición (Ciclo hamiltoniano). Un ciclo hamiltoniano es una sucesión $(v_1, v_2, \ldots, v_n)$ que contiene cada vértice de V exactamente una vez, tal que $\{v_i, v_{i+1}\} \in E$ para $i = 1, \ldots, n-1$ y, además, $\{v_n, v_1\} \in E$ (el cierre). Tiene exactamente n aristas. Decidir si un grafo tiene uno es un problema NP-completo [7], por eso se usan metaheurísticas.

Definición (Permutación). Una permutación de V es un ordenamiento de sus n vértices en el que ninguno se repite ni falta. Existen $n!$ permutaciones; si se fija el vértice inicial y se ignora el sentido del recorrido, quedan $(n-1)!/2$ ciclos distintos posibles.

Definición (Espacio de búsqueda y factibilidad). El espacio de búsqueda es el conjunto de todas las soluciones candidatas que el algoritmo puede generar con la representación elegida. Una candidata es factible si cumple todas las reglas del problema; en este caso, si es un ciclo hamiltoniano.

3.2. Tema general: representación del espacio de búsqueda

La representación decide qué estructuras puede producir el algoritmo y, por lo tanto, qué tan fácil es caer en soluciones ilegales. Larrañaga et al. [4] muestran, para el problema del viajero (TSP), que la misma metaheurística cambia mucho de comportamiento según la codificación y los operadores. Hay que representar dos cosas distintas: el grafo (los datos del problema) y la ruta (la solución candidata).

3.2.1. Cómo guardar el grafo

Matriz de adyacencia. Es una tabla A de $n \times n$ con $A[u][v] = 1$ si la arista $\{u, v\}$ existe y 0 si no. Consultar si dos vértices están conectados cuesta $O(1)$, pero la memoria es $O(n^2)$ aunque el grafo tenga pocas aristas [6].

Lista de adyacencia. Para cada vértice se guarda la lista de sus vecinos. Usa memoria $O(n+m)$ y permite recorrer los vecinos de un vértice en tiempo proporcional a su grado, que es justo lo que hace una hormiga al elegir el siguiente paso. Saber si existe una arista concreta cuesta $O(\text{grado})$ [6].

Tabla 3.1: Comparación de representaciones del grafo.

	Matriz de adyacencia	Lista de adyacencia
Memoria	$O(n^2)$	$O(n + m)$
¿Existe $\{u, v\}$?	$O(1)$	$O(\text{grado}(u))$
Recorrer vecinos de u	$O(n)$	$O(\text{grado}(u))$
Adecuada para	grafos densos o pequeños	grafos dispersos

Dimensión real del FHCP. Las 1001 instancias del FHCP tienen entre 66 y 9528 vértices [8, 9]. Para $n = 9528$, una matriz tiene $9528^2 \approx 9.08 \times 10^7$ celdas: unos 91 MB si se usa un byte por celda (unos 11 MB si se usa un bit). Es manejable, pero desperdicia memoria si el grafo es disperso.

Propuesta: Usar listas de adyacencia para recorrer vecinos y, adicionalmente, un conjunto de aristas (tabla hash de pares ordenados $(\text{mín}, \text{máx})$) para verificar en $O(1)$ el cierre y la validez de una ruta. Cuestión pendiente:

Medir el grado medio de las instancias al descargarlas para confirmar que son dispersas.

Formato de los archivos. El FHCP usa el formato HCP [9]: una cabecera, la lista de aristas (una por línea) y un cierre. Las soluciones usan el mismo estilo con `TYPE : TOUR` y una `TOUR_SECTION` con los vértices en el orden del ciclo, también terminada en `-1`. Esquema (los valores son ilustrativos):

```
NAME : graph1
COMMENT : ...
TYPE : HCP
DIMENSION : 66
EDGE_DATA_FORMAT : EDGE_LIST
EDGE_DATA_SECTION
1 2
...
-1
EOF
```

El ejemplo oficial usa identificadores desde 1. El lector deberá comprobar el rango indicado por DIMENSION y preservar los identificadores al exportar.[9]

3.2.2. Cómo representar una ruta

Requisito: La práctica pide estructuras que representen permutaciones de los vértices sin generar «cromosomas» ilegales, y que cada programa devuelva un arreglo de vértices como solución propuesta.

Propuesta: La representación más directa es la permutación de vértices (path representation en [4]): un arreglo de longitud n como $[1,\ 7,\ 6,\ 5,\ 4,\ 3,\ 2]$, que se lee como el recorrido $1 \to 7 \to 6 \to 5 \to 4 \to 3 \to 2 \to 1$. Garantiza por construcción que cada vértice aparece una sola vez.

Otras codificaciones descritas en [4] y útiles para el equipo:

- Adyacencia: la posición i guarda el sucesor del vértice i.
- Ordinal: cada gen indica una posición en una lista de vértices restantes; cualquier cruza de un punto produce una ruta válida.
- Llaves aleatorias (random keys) [11]: un vector de números reales que se ordena para obtener la permutación. Es la puerta de entrada natural para algoritmos continuos como DE y PSO (integrantes 4 y 5).

3.2.3. Por qué una permutación válida puede no ser un ciclo

Una permutación cumple dos de las cuatro reglas del ciclo hamiltoniano: están todos los vértices y ninguno se repite. Pero no garantiza las otras dos:

1. que cada par de vértices consecutivos esté unido por una arista del grafo, y
2. que exista la arista de cierre entre el último y el primero.

En el TSP clásico el grafo es completo y toda permutación es una ruta; en el ciclo hamiltoniano el grafo es incompleto (y en el FHCP, deliberadamente engañoso [8]), así que la mayoría de las permutaciones usa aristas inexistentes.

3.2.4. Cómo tratar rutas que no cumplen las reglas

Coello Coello [5] clasifica las técnicas de manejo de restricciones; las tres que aplican aquí:

1. Evitar (representación y operadores que preservan factibilidad): la ruta solo avanza por aristas existentes. ACO lo hace de forma natural, porque cada hormiga elige entre los vecinos del vértice actual.

2. Penalizar: se aceptan permutaciones arbitrarias y se mide cuánto les falta. Propuesta: Función de costo

$$f(\pi) = \sum_{i=1}^{n} \big[\{\pi_i, \pi_{(i \bmod n)+1}\} \notin E\big],$$

   es decir, el número de aristas faltantes incluido el cierre. Vale 0 si y solo si π es un ciclo hamiltoniano, y sirve como objetivo común para comparar GA, SA, DE y PSO.

3. Reparar: transformar una candidata ilegal en una mejor. Por ejemplo, la rotación de Pósa [10] toma un camino $v_1 \dots v_k$ cuyo extremo v_k es vecino de algún v_i, y lo reescribe como $v_1 \dots v_i\, v_k v_{k-1} \dots v_{i+1}$, que tiene un extremo nuevo desde el cual intentar extender o cerrar.

Comprobación de una ruta (la misma para todos los algoritmos). Dada $\pi = (\pi_1, \dots, \pi_n)$: (a) su longitud es n; (b) no hay repetidos (un conjunto de tamaño n); (c) cada par consecutivo es arista; (d) $\{\pi_n, \pi_1\}$ es arista. Cuesta $O(n)$ con el conjunto de aristas.

3.3. Algoritmo: Optimización por Colonia de Hormigas (ACO)

3.3.1. Idea

Las hormigas reales depositan feromona al caminar; los caminos cortos se recorren más veces por unidad de tiempo, acumulan más feromona y atraen a más hormigas. ACO imita este mecanismo: un conjunto de agentes construye soluciones paso a paso y se comunica de forma indirecta (estigmergia) a través de valores numéricos asociados a las aristas [1, 3]. La primera versión fue el Ant System (AS) [1]; luego aparecieron variantes como Ant Colony System (ACS) [2] y MAX–MIN Ant System (MMAS) [3, 13].

3.3.2. Construcción de rutas sin repetir vértices

Cada hormiga k empieza en un vértice y mantiene una memoria (lista tabú) de los vértices ya visitados. Estando en i, sus opciones son los vecinos aún no visitados N_i^k. En AS elige $j \in N_i^k$ con probabilidad [1, 3]

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \, \eta_{ij}^\beta}{\sum_{l \in N_i^k} \tau_{il}^\alpha \, \eta_{il}^\beta},$$

donde τ_{ij} es la feromona de la arista, η_{ij} es la información heurística, y α, β ponderan una contra otra. Como solo se consideran vecinos no visitados, la hormiga nunca repite vértices ni usa aristas inexistentes: la representación evita repeticiones y pasos inexistentes; aún puede terminar en un camino parcial o sin cierre, que no es una solución válida.

3.3.3. Feromonas: refuerzo y evaporación

La feromona τ_{ij} es la memoria colectiva: cuánto ha convenido usar la arista $\{i, j\}$ en el pasado. Tras cada iteración [1, 3]:

$$\tau_{ij} \leftarrow (1 - \rho)\, \tau_{ij} + \sum_k \Delta\tau_{ij}^k.$$

- Evaporación ($0 < \rho \leq 1$): todas las aristas pierden una fracción de su feromona. Evita que la colonia se quede para siempre con las primeras rutas encontradas.

- Depósito ($\Delta\tau_{ij}^k$): las hormigas refuerzan las aristas que usaron, en proporción a la calidad de su ruta. En el TSP, $\Delta\tau = Q/L_k$ (L_k = longitud del recorrido).

Variantes: en ACS solo la mejor ruta deposita y además hay una actualización local que reduce la feromona de las aristas recién usadas para diversificar [2]; en MMAS solo deposita la mejor hormiga y τ se limita a $[\tau_{\text{mín}}, \tau_{\text{máx}}]$ para evitar el estancamiento [13].

3.3.4. Información del grafo para elegir el siguiente paso

En el TSP la heurística es $\eta_{ij} = 1/d_{ij}$ (preferir ciudades cercanas). En el ciclo hamiltoniano no hay distancias, así que hay que usar la estructura del grafo. Propuesta: Una opción natural es la regla de Warnsdorff (preferir el vecino con menos vecinos libres):

$$\eta_{ij} = \frac{1}{\text{máx}\big(1,\ |N(j) \setminus \text{visitados}|\big)}.$$

La intuición: un vértice con pocos vecinos libres corre peligro de quedar aislado, así que conviene visitarlo pronto. Otras señales útiles: si un vértice tiene grado 2, sus dos aristas son obligatorias en cualquier ciclo hamiltoniano, y conviene evitar los movimientos que dejen algún vértice libre sin vecinos libres. Cuestión pendiente: La regla de Warnsdorff proviene del recorrido del caballo; no encontramos todavía una fuente que la evalúe con ACO sobre el FHCP.

3.3.5. Callejones sin salida y ciclos que no cierran

Al trabajar sobre un grafo incompleto aparecen dos fallas que no existen en el TSP:

1. Callejón sin salida: la hormiga llega a un vértice cuyos vecinos ya fueron visitados, pero todavía faltan vértices. Su ruta es un camino de $k < n$ vértices.

2. Camino hamiltoniano sin cierre: visita los n vértices, pero el último no es vecino del primero.

Opciones para tratarlas:

- Propuesta: Puntuar la ruta parcial con el número de aristas válidas recorridas (incluido el cierre si existe) dividido entre n, y depositar feromona en proporción a ese valor. Así una hormiga casi exitosa aporta información útil.
- Propuesta: Reparar con rotaciones de Pósa [10] antes de evaluar: si el extremo tiene un vecino ya visitado, rotar el camino y seguir extendiendo o intentar cerrar.
- Propuesta: Retroceso limitado (backtracking): deshacer los últimos pasos y probar otro vecino, con un tope de intentos para no convertirlo en fuerza bruta.
- Wagner y Bruckstein [12] propusieron Hamiltonian(t), un proceso de una sola hormiga que marca vértices y elige según las marcas vecinas; muestran que los ciclos hamiltonianos son ciclos límite de ese proceso. Es un antecedente directo de hormigas aplicadas a este problema.

3.4. Ejemplo didáctico con siete vértices

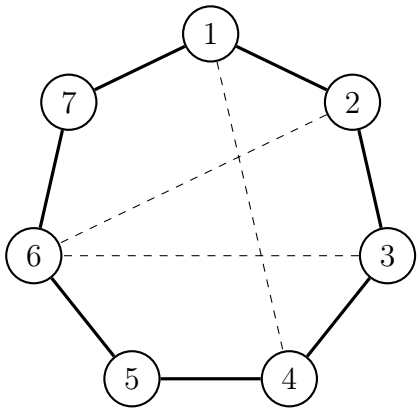


Figura 3.1: Grafo de ejemplo con $n = 7$ y $m = 10$. En azul, el único ciclo hamiltoniano; en gris discontinuo, las tres aristas «trampa» $\{1, 4\}$, $\{2, 6\}$ y $\{3, 6\}$.

Representación. Lista de adyacencia: $1 : \{2, 4, 7\}$, $2 : \{1, 3, 6\}$, $3 : \{2, 4, 6\}$, $4 : \{1, 3, 5\}$, $5 : \{4, 6\}$, $6 : \{2, 3, 5, 7\}$, $7 : \{1, 6\}$.

Razonamiento estructural. Los vértices 5 y 7 tienen grado 2, así que las aristas $\{4,5\}$, $\{5,6\}$, $\{6,7\}$ y $\{7,1\}$ son obligatorias. Entonces el vértice 6 ya usa dos aristas (hacia 5 y 7) y quedan descartadas $\{2,6\}$ y $\{3,6\}$. Falta unir 1 con 4 pasando por 2 y 3: solo es posible como 1–2–3–4. El ciclo hamiltoniano es único: 1–2–3–4–5–6–7–1 (lo confirmamos enumerando las 6! permutaciones que empiezan en 1: solo esta y su recorrido inverso tienen costo 0). Este es el mismo tipo de estructura que vuelve difíciles a las instancias del FHCP: pocas aristas correctas escondidas entre muchas tentadoras [8].

Costo de permutaciones (función de penalización).

Permutación	Aristas faltantes	Motivo
[1, 4, 3, 6, 7, 2, 5]	3	faltan $\{7,2\}$, $\{2,5\}$ y el cierre $\{5,1\}$
[2, 3, 4, 1, 7, 6, 5]	1	todas existen salvo el cierre $\{5,2\}$
[1, 2, 3, 4, 5, 6, 7]	0	ciclo hamiltoniano

Primer paso de una hormiga. Parámetros $\alpha = 1$, $\beta = 2$, $\tau_0 = 1$ en todas las aristas y la heurística de Warnsdorff. Desde el vértice 1, los vecinos libres de 2, 4 y 7 son 2, 2 y 1, respectivamente, así que $\eta = (\frac{1}{2}, \frac{1}{2}, 1)$ y

$$p_{12} = \frac{0.25}{0.25 + 0.25 + 1} = 0.167, \quad p_{14} = 0.167, \quad p_{17} = 0.667.$$

La heurística empuja hacia el vértice 7, de grado 2, que es correcto. Desde 7 el único vecino libre es 6. En 6 (visitados $\{1,7,6\}$) los candidatos son 2, 3 y 5 con $\eta = (1, \frac{1}{2}, 1)$, de modo que $p_{62} = 0.444$, $p_{63} = 0.111$, $p_{65} = 0.444$.

Tres hormigas de la primera iteración.

- Hormiga A: $1-7-6-2-3-4-5$. Visita los 7 vértices, pero $\{5,1\}$ no existe: camino hamiltoniano sin cierre. 6 aristas válidas.
- Hormiga B: $1-4-3-6-7$. En 7 sus vecinos (1 y 6) ya fueron visitados y faltan 2 y 5: callejón sin salida. 4 aristas válidas.
- Hormiga C: $1-7-6-5-4-3-2-1$. Ciclo hamiltoniano: 7 aristas válidas y cierre correcto.

Comprobación de C: longitud 7, sin repetidos, las aristas $\{1,7\}, \{7,6\}, \{6,5\}, \{5,4\}, \{4,3\}, \{3,2\}$ existen y el cierre $\{2,1\}$ también.

Actualización de feromonas con $\rho = 0.5$ y depósito $\Delta\tau = (\text{aristas válidas})/n$: A deposita $6/7 = 0.857$, B $4/7 = 0.571$ y C 1.

Arista	1-2	1-4	1-7	2-3	2-6	3-4	3-6	4-5	5-6	6-7
τ nueva	1.500	1.071	2.357	2.357	1.357	2.929	1.071	2.357	1.500	2.929

En la segunda iteración, desde 6 (tras $1-7$) las probabilidades pasan a $p_{62} = 0.434$, $p_{63} = 0.086$, $p_{65} = 0.480$: el aprendizaje favorece la arista correcta, pero lentamente, porque la hormiga A también reforzó la trampa $\{2, 6\}$. Si se usa un esquema elitista como MMAS, en el que solo deposita la mejor ruta (C), entonces $\tau_{62} = 0.5$, $\tau_{63} = 0.5$ y $\tau_{65} = 1.5$, y las probabilidades quedan en 0.235, 0.059 y 0.706. Lección: en grafos engañosos importa mucho quién deposita feromona.

3.5. Pseudocódigo y ajustes principales

Algorithm 1 ACO para el ciclo hamiltoniano (variante estilo MMAS)

Require: Grafo $G = (V, E)$ como listas de adyacencia; parámetros $m, \alpha, \beta, \rho, \tau_{\text{mín}}, \tau_{\text{máx}}$, límite de iteraciones T

Ensure: Arreglo de n vértices que forma un ciclo hamiltoniano, o la mejor ruta parcial

$\tau_{ij} \leftarrow \tau_{\text{máx}}$ para toda $\{i, j\} \in E$
mejor $\leftarrow$ ruta vacía
for $t = 1$ to T **do**
  for $k = 1$ to m **do**
    elegir vértice inicial s al azar; $R_k \leftarrow [s]$; *visitados* $\leftarrow \{s\}$
    while $|R_k| < n$ **do**
      $i \leftarrow$ último vértice de R_k
      $N \leftarrow$ vecinos de i no visitados
      if $N = \emptyset$ **then**
        intentar reparar con rotación de Pósa; si no se puede, break
      else
        elegir $j \in N$ con probabilidad $\propto \tau_{ij}^{\alpha} \eta_{ij}^{\beta}$
        añadir j a R_k y a *visitados*
      end if
    end while
    $q_k \leftarrow$ (aristas válidas de R_k, incluido el cierre si existe)$/n$
    if $q_k = 1$ y R_k pasa el verificador completo **then return** R_k ▷ ciclo hamiltoniano encontrado
    end if
    if $q_k >$ calidad de *mejor* **then** *mejor* $\leftarrow R_k$
    end if
  end for
  $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$ para toda arista ▷ evaporación
  $\tau_{ij} \leftarrow \tau_{ij} + q_{mejor}$ para cada arista de *mejor* ▷ depósito
  acotar τ_{ij} al intervalo $[\tau_{\text{mín}}, \tau_{\text{máx}}]$
end for
return *mejor*

Tabla 3.2: Ajustes principales y valores de partida. Los valores de referencia son los que Dorigo y Stützle [3] recomiendan para el TSP; hay que calibrarlos para el FHCP.

Parámetro	Efecto	Referencia (TSP)
m (hormigas)	Más hormigas exploran más por iteración, pero cuestan más tiempo.	$m = n$ (AS, MMAS); $m = 10$ (ACS)
α	Peso de la feromona (lo aprendido). Muy alto: convergencia prematura.	1
β	Peso de la heurística (la estructura del grafo).	2 a 5
ρ	Evaporación. Alta: olvida rápido y explora; baja: explota lo aprendido.	0.5 (AS), 0.1 (ACS), 0.02 (MMAS)
$\tau_{\text{mín}}, \tau_{\text{máx}}$	Evitan que una arista quede con probabilidad casi 0 o casi 1.	MMAS
q_0	(ACS) probabilidad de elegir directamente el mejor vecino.	0.9
T	Límite de iteraciones o de tiempo; necesario para la comparación.	—

3.6. Aplicación al ciclo hamiltoniano: opciones, dificultades y dudas

3.6.1. Opciones de diseño

1. Propuesta: Grafo de construcción = grafo del problema: las hormigas solo se mueven por aristas reales, así que no repiten vértices ni recorren aristas inexistentes; las fallas se reducen a «no llegó a todos» o «no cerró».

2. Propuesta: Calidad = fracción de aristas válidas. Es equivalente a $1 - f(\pi)/n$ con la función de penalización presentada en este capítulo, para permutaciones completas. En rutas parciales, esta puntuación es solo una guía interna; la comparación entre métodos debe usar éxito verificado y presupuesto.

3. Propuesta: Variante MMAS o ACS: el ejemplo muestra que el depósito de todas las hormigas refuerza trampas; un esquema elitista con límites de feromona es más apropiado.

4. Propuesta: Preprocesamiento: fijar las aristas obligatorias de los vértices de grado 2 antes de lanzar la colonia.

5. Propuesta: Búsqueda local: combinar ACO con rotaciones de Pósa o 2-opt restringido a aristas existentes, como se hace con ACO+búsqueda local en el TSP [3].

3.6.2. Dificultades previstas

- Las instancias del FHCP están construidas para engañar a heurísticas estándar [8]; muchas aristas «atractivas» no pertenecen a ningún ciclo.
- Si el número de hormigas es $m = n$ y el grafo tiene miles de vértices, una iteración cuesta del orden de n^2 pasos; para $n \approx 9500$ son unos 9×10^7 pasos.
- La feromona mide aristas, pero el problema es global: una arista puede estar en rutas casi completas y aun así no pertenecer a ningún ciclo hamiltoniano.
- Las rutas que fallan muy pronto dan poca información; la retroalimentación es escasa.
- Como referencia de dificultad, el ganador del concurso FHCP resolvió 985 de las 1001 instancias [9]; no se espera que una metaheurística simple las resuelva todas.

3.6.3. Dudas pendientes

1. Cuestión pendiente: ¿El profesor espera trabajar con las 1001 instancias o con un subconjunto (por ejemplo, las más pequeñas)?
2. Cuestión pendiente: ¿Qué límite de tiempo o de iteraciones usaremos para que la comparación sea justa entre los seis algoritmos?
3. Cuestión pendiente: ¿Se permite combinar ACO con búsqueda local o reparación, o se pide la versión «pura» del algoritmo?
4. Cuestión pendiente: ¿Cómo aplicar precisión, recall y F1 (tema del integrante 4) a un problema donde todas las instancias son hamiltonianas? Una propuesta es tratar «encontró el ciclo» como predicción positiva y comparar contra la solución oficial.
5. Cuestión pendiente: ¿Acordamos en el equipo una única función de costo (aristas faltantes) para todos los algoritmos?

Capítulo 4

Módulo evolutivo, heurísticas y colonia de abejas

4.1. El problema que se quiere resolver

Un grafo $G = (V, E)$ reúne vértices V y aristas E que indican qué pares están conectados. Un ciclo hamiltoniano recorre cada vértice exactamente una vez y vuelve al inicial por una arista existente. Si hay n vértices, una ruta puede escribirse $R = (v_1, v_2, \ldots, v_n, v_1)$: el primer vértice aparece de nuevo al final solamente para cerrar el ciclo. La pregunta de decisión es si existe alguna ruta con esas propiedades.[14] No basta con escribir una permutación de vértices: deben existir todas sus aristas consecutivas, incluida (v_n, v_1).

El problema del viajante (TSP) agrega costos a las aristas y busca un ciclo de costo mínimo; el problema hamiltoniano planteado aquí busca existencia. Esta distinción cambia la evaluación: una ruta barata no sirve si usa aristas ausentes. El conjunto FHCP es una colección de instancias difíciles del problema hamiltoniano; el ejemplo pequeño de la sección 4.4 fue inventado para explicar el método y no pertenece a ese conjunto.[15]

Hay que separar también camino hamiltoniano y ciclo hamiltoniano. Un camino visita todos los vértices sin repetirlos, pero no necesita regresar al punto de partida. Un ciclo exige además una última arista que conecte el final con el inicio. Esa condición de cierre es fácil de olvidar cuando la ruta se guarda como una lista sin repetir el primer vértice al final. Para comprobarla, la implementación debe tratar la lista como circular.

4.2. Módulo evolutivo y heurísticas

4.2.1. Qué es una heurística

Una heurística es una regla práctica que usa información disponible para orientar una búsqueda hacia opciones prometedoras. Por ejemplo, al extender una ruta parcial puede preferirse un vecino que deje más posibilidades de continuar. Eso ahorra exploración en muchos casos, pero una preferencia no demuestra que exista una solución ni que se haya encontrado la mejor. En búsqueda informada, una función heurística suele estimar el costo o la distancia restante a la meta; en búsqueda de poblaciones también se usan puntuaciones para favorecer candidatos.[16]

Conviene distinguir tres niveles: representación (cómo se codifica una posible ruta), evaluación (qué tan prometedora parece) y comprobación (si satisface exactamente la definición de ciclo). Una heurística guía el trabajo; el comprobador da el veredicto sobre una ruta concreta.

Ejemplo de orientación. Si una ruta parcial termina en C y puede pasar a D o a E, una regla podría preferir el vértice que conserve más conexiones hacia los aún no visitados. Esa regla usa la estructura del grafo para decidir qué probar primero. Puede equivocarse: elegir una continuación que parece buena ahora puede impedir el regreso al inicio después. Por ello, una heurística se evalúa por su utilidad práctica para organizar intentos, no por su capacidad de certificar una respuesta.

En métodos de población aparece otra distinción: exploración significa probar regiones nuevas del espacio de rutas, mientras que explotación significa mejorar rutas ya prometedoras. Si solo se explota, varias soluciones pueden quedarse cerca del mismo resultado mediocre; si solo se explora, no se aprovecha lo aprendido. La colonia artificial de abejas combina ambos comportamientos mediante observadoras que se concentran en fuentes buenas y exploradoras que renuevan fuentes estancadas.[20, 21]

4.2.2. Cómo representar y modificar una posible solución

Para un grafo con vértices A, B, C, D, E, F, puede guardarse la permutación (A, B, C, D, E, F) e interpretar que después de F se intenta regresar a A. Fijar A al principio evita copias debidas a rotaciones de la misma ruta. En grafos no dirigidos, una ruta y su reverso describen el mismo ciclo; eliminar

esa duplicación es opcional.

Una solución candidata puede generarse con una permutación aleatoria. Se modifica mediante un intercambio de dos posiciones, la inversión de un tramo o la inserción de un vértice en otra posición. Estas operaciones conservan la propiedad de visitar una vez cada vértice, pero pueden crear o destruir aristas válidas. Por eso se vuelve a evaluar y comprobar cada cambio. Para un recorrido parcial, la información útil puede incluir aristas disponibles, vértices sin visitar y posibilidades de cerrar el ciclo. Para una permutación completa, puede incluir el número y la localización de enlaces ausentes.

Una permutación de n vértices ofrece un espacio de búsqueda grande: fijar el inicio deja $(n-1)!$ órdenes posibles. En un grafo no dirigido, leer un ciclo en sentido inverso representa el mismo conjunto de aristas, por lo que incluso entre esos órdenes hay duplicados. La codificación debe preservar una regla sencilla: cada vértice aparece exactamente una vez en la lista y el regreso se agrega al interpretarla. Así, la búsqueda se concentra en mejorar las conexiones en lugar de corregir vértices repetidos.

Cambio	Ejemplo y efecto sobre (A, B, C, D, E, F)
Intercambio	Cambiar C y E produce (A, B, E, D, C, F). Se revisan las conexiones alrededor de las dos posiciones.
Inversión	Invertir el tramo C, D, E produce (A, B, E, D, C, F). Se revisan los extremos y la orientación interna del tramo.
Inserción	Mover E antes de C produce (A, B, E, C, D, F). Se revisan las conexiones de la posición original y de la nueva.

Los dos primeros cambios coinciden en este ejemplo corto, pero son operadores distintos en general: el intercambio afecta exactamente dos posiciones, mientras que la inversión puede alterar todo un tramo. En este trabajo el pseudocódigo elige solo intercambio para que la adaptación de ABC sea inequívoca.

4.2.3. Función de evaluación, fitness y validez

Sea $R = (v_1, \ldots, v_n)$ una permutación y definamos $v_{n+1} = v_1$. Una evaluación simple propuesta para este documento cuenta aristas inexistentes:

$$b(R) = \sum_{i=1}^{n} \mathbf{1}\big[(v_i, v_{i+1}) \notin E\big], \qquad F(R) = \frac{1}{1 + b(R)}. \tag{4.1}$$

Aquí b se minimiza; F es el fitness, una calidad positiva que se maximiza. Si hay dos enlaces faltantes, $F = 1/3$; si no falta ninguno, $F = 1$. Esta puntuación hace comparables rutas incompletas y permite elegir probabilísticamente entre ellas. No es la única función posible y no implica distancias, pues el problema descrito no da pesos a las aristas.

La validez se comprueba de forma explícita: (1) la ruta contiene exactamente los n vértices distintos del grafo; (2) cada par consecutivo es una arista; y (3) también existe la arista del último al primero. Si la representación garantiza una permutación, $b(R) = 0$ equivale a validez para un grafo simple no dirigido, pero conviene conservar un verificador independiente para detectar errores de representación o de implementación. Una ruta con $b = 1$ es casi válida, nunca una solución.

La expresión $\mathbf{1}[P]$ vale 1 cuando la proposición P es verdadera y 0 en caso contrario. Por eso $b(R)$ revisa exactamente n conexiones: las $n - 1$ parejas consecutivas de la lista y la conexión de regreso. El valor $F(R)$ siempre es positivo y decrece al aumentar los enlaces ausentes. Se eligió esta transformación porque la selección por ruleta necesita pesos no negativos y porque resulta fácil explicar por qué una fuente con menos fallos recibe más atención.[21]

Ejemplo mínimo de la diferencia. Si una ruta contiene todos los vértices y sus primeras $n - 1$ conexiones existen, pero falta la última arista hacia el inicio, entonces $b = 1$ y $F = 1/2$. El fitness expresa que la ruta está cerca del objetivo según esta medida; el verificador responde "inválida". Dos rutas distintas con $b = 1$ reciben igual nota aunque una sea más fácil de reparar. Esta igualdad produce una meseta y explica por qué el operador de vecindad y las exploradoras importan tanto como la fórmula de fitness.

4.2.4. Algoritmo genético (GA) como referencia

Un algoritmo genético mantiene una población de candidatos, llamados individuos. Cada individuo codifica una ruta. En cada generación se evalúan los individuos, se seleccionan padres favoreciendo los mejores, se hace cruce para combinar información de dos padres y se aplica mutación para introducir variaciones. Después se forma una nueva población y se repite. Esta es la estructura general que la práctica menciona como referencia y que el equipo tendrá que conocer antes de programarla.[17, 18]

En rutas representadas como permutaciones, un cruce ingenuo puede repetir vértices y omitir otros. Por ejemplo, los padres (B, C, D, E, F) y (C, D, B, E, F), cortados después de dos posiciones y pegados sin cuidado,

pueden producir (B, C, B, E, F): B se repite y falta D. Debe usarse un cruce que preserve permutaciones (por ejemplo, uno basado en orden o PMX), o reparar el descendiente.[19] Una mutación por intercambio también preserva la permutación. La selección puede hacerse por torneo o por ruleta; la decisión concreta depende del diseño de GA que adopte el equipo.

El ciclo básico de GA se entiende mejor como una secuencia de responsabilidades:

1. Inicializar: crear varias rutas diferentes para no depender de un solo punto de partida.

2. Evaluar: asignar fitness a cada individuo; el verificador de validez puede detener la búsqueda si aparece un ciclo.

3. Seleccionar padres: dar mayor oportunidad de reproducción a rutas mejor puntuadas, sin tener que copiar únicamente a la mejor.

4. Cruzar y mutar: combinar información de padres con un operador que conserve permutaciones y modificar algunas rutas para crear variedad.

5. Reemplazar: decidir qué individuos entran en la siguiente generación; opcionalmente conservar al mejor para no perder un avance.

6. Repetir o detener: continuar hasta hallar un ciclo o agotar el presupuesto definido para la búsqueda.

La selección por torneo compara unos pocos individuos elegidos al azar y toma al mejor del grupo; la ruleta asigna probabilidades proporcionales a un fitness positivo. El cruce intenta heredar segmentos u órdenes útiles de dos padres. La mutación introduce cambios que los padres no aportaron. Ninguna de estas operaciones, por sí sola, garantiza que las aristas de la ruta final existan: siempre hace falta evaluar y verificar.[17, 18, 19]

GA y ABC comparten la idea de mantener varias soluciones y puntuarlas. Su mecánica principal difiere: GA recombina padres y muta descendientes; ABC asigna una fuente de alimento a cada abeja empleada, mejora fuentes por vecindad, recluta observadoras según calidad y reemplaza fuentes estancadas.[17, 18, 20]

4.3. Colonia artificial de abejas: versión elegida

4.3.1. Origen y alcance de la adaptación

La versión de referencia es Artificial Bee Colony (ABC) de Derviş Karaboğa (2005). Su propuesta original trataba optimización numérica: cada fuente de alimento era un vector de números reales. Aquí se conserva su ciclo de fases y se propone una adaptación discreta por intercambio para permutaciones de vértices. No se usa la fórmula continua de perturbación de coordenadas sobre rutas, porque produciría valores que no son permutaciones. Tampoco se mezclan mecanismos de otros algoritmos de abejas. Existen trabajos académicos que adaptan ABC a problemas combinatorios y a recorridos, lo que respalda la necesidad de definir operadores discretos; el operador y el fitness exactos de este documento son decisiones didácticas.[20, 21, 22]

Término ABC	Interpretación en esta propuesta
Fuente de alimento	Una permutación de todos los vértices, con regreso implícito al inicial.
Néctar o calidad	El fitness $F(R)$ de la ecuación (4.1).
Abeja empleada	Trabaja sobre una fuente asignada e intenta mejorarla mediante un intercambio.
Abeja observadora	Elige una fuente con mayor probabilidad cuanto mayor sea su fitness; también intenta mejorarla.
Abeja exploradora	Sustituye una fuente que acumula demasiados intentos sin mejora por una nueva permutación.

4.3.2. Qué ocurre en un ciclo de ABC

En la inicialización se crean fuentes independientes y se calcula su calidad. La analogía con una fuente de alimento no significa que haya abejas moviéndose físicamente por el grafo: cada fuente es una estructura de datos que describe una ruta. Se asocia una abeja empleada a cada fuente. En la versión original de ABC, las abejas empleadas y las observadoras forman los dos grupos principales, y una empleada puede convertirse en exploradora cuando su fuente se abandona.[20]

Durante la fase de empleadas, cada fuente recibe al menos un intento local. En la adaptación propuesta, la empleada intercambia dos vértices de

su ruta y compara el fitness nuevo con el anterior. Si el intento mejora, conserva la vecina. Si no mejora, la fuente permanece igual y el contador registra el fracaso. Esta fase asegura que incluso una fuente poco popular tenga oportunidad de mejorar.

En la fase de observadoras, se publica la calidad de las fuentes y se eligen con las probabilidades de la ecuación (4.2). Una fuente de alta calidad puede recibir varias observadoras en el mismo ciclo y otra puede no recibir ninguna. Cada observadora intenta su propio cambio local y aplica la misma decisión voraz. Aquí aparece la explotación: se invierten más intentos en rutas que ya parecen prometedoras.[21]

Por último, la fase de exploradoras revisa el estancamiento. Una fuente que acumuló `limit` intentos sin mejora deja de ocupar un lugar de la colonia y se reemplaza por otra ruta creada desde cero. Esto reintroduce diversidad. El algoritmo guarda la mejor solución histórica aparte, porque abandonar la fuente actual no debería borrar un resultado útil anterior. El ciclo completo se repite hasta hallar un ciclo válido o agotar el presupuesto de búsqueda.[20, 21]

4.3.3. Selección y abandono

Si existen SN fuentes y todas tienen fitness positivo, la probabilidad propuesta de que una observadora elija la fuente i es

$$p_i = \frac{F(R_i)}{\sum_{j=1}^{SN} F(R_j)}. \tag{4.2}$$

Una fuente buena tiene más oportunidades, pero una de menor calidad conserva probabilidad no nula. Después de crear una vecina R', se usa selección voraz: se reemplaza R solo si $F(R') > F(R)$; en caso contrario se conserva R y aumenta su contador de intentos sin mejora. En esta propuesta, un empate también cuenta como intento sin mejora. Si el contador alcanza `limit`, la fuente se abandona y una exploradora crea una nueva permutación; su contador vuelve a cero. Una mejora también reinicia el contador. Se guarda aparte la mejor ruta vista hasta el momento para no perderla al reemplazar fuentes. El parámetro `limit` y estas fases provienen del ABC de referencia; los detalles de empate y operador discreto son decisiones explícitas de la adaptación.[20, 21]

4.3.4. Pseudocódigo de la adaptación propuesta

1. Recibir el grafo $G = (V, E)$, el número de fuentes SN, el número máximo de ciclos $C_{\text{máx}}$ y `limit`. Usar SN empleadas y SN observadoras.

2. Crear SN permutaciones aleatorias de V con el vértice inicial fijo; para cada una calcular b y F, y poner su contador en cero. Guardar la mejor encontrada.

3. Si alguna permutación pasa la comprobación exacta de ciclo hamiltoniano, devolverla.

4. Para cada abeja empleada: elegir dos posiciones distintas, excepto la inicial, e intercambiar sus vértices. Evaluar la vecina. Si mejora estrictamente, reemplazar la fuente, reiniciar su contador, actualizar la mejor y comprobar si ya es un ciclo; en otro caso, aumentar el contador.

5. Calcular las probabilidades p_i con la ecuación (4.2).

6. Para cada abeja observadora: elegir una fuente según p_i, crear otra vecina mediante intercambio y aplicar la misma regla voraz, el mismo contador y la comprobación de ciclo al aceptar una mejora.

7. Para cada fuente cuyo contador haya alcanzado `limit`: activar la fase exploradora, sustituirla por una nueva permutación aleatoria, evaluarla, reiniciar su contador y comprobar si ya es un ciclo.

8. Actualizar la mejor ruta vista y comprobarla con el verificador exacto. Si es válida, devolverla.

9. Repetir los pasos 4 a 8 hasta $C_{\text{máx}}$ o hasta otro límite de recursos acordado.

10. Si no se encontró un ciclo, informar "no encontrado con este presupuesto de búsqueda". El agotamiento del tiempo de una heurística no demuestra que el grafo carezca de ciclo.

Los ajustes principales son SN (cuántas regiones se exploran en paralelo), $C_{\text{máx}}$ (presupuesto de búsqueda), `limit` (cuándo abandonar una fuente estancada) y el operador de vecindad (qué cambios puede intentar cada abeja). Una población muy pequeña puede perder diversidad; una muy grande cuesta más evaluaciones. Un `limit` bajo promueve exploración frecuente;

uno alto permite insistir más sobre una fuente. No hay valores universales: se ajustarían después con experimentos reproducibles, fuera del alcance de esta investigación.[20, 21]

Ajuste	Si se reduce	Si se aumenta
SN	Menos fuentes y menos diversidad inicial; cada ciclo cuesta menos.	Más rutas simultáneas y más evaluaciones por ciclo.
$C_{\text{máx}}$	La búsqueda acaba antes, incluso si estaba progresando.	Se permiten más intentos, con mayor costo total.
`limit`	Las fuentes se abandonan con rapidez y se explora más.	Se insiste más tiempo en cada fuente, incluso si está estancada.
Vecindad	Un cambio muy local es barato, pero puede avanzar lentamente.	Cambios más amplios recorren más posibilidades, aunque pueden destruir conexiones útiles.

Cada evaluación de $b(R)$ puede revisar las n parejas de la ruta una vez. Si las aristas del grafo se consultan rápidamente, ese recuento crece linealmente con n. Un intercambio solo altera las conexiones que tocan las dos posiciones cambiadas, de modo que una implementación posterior podría actualizar el valor localmente; para comprender y verificar el método resulta más transparente recalcular las n parejas. Esto es una observación de diseño, no un requisito de la práctica.

4.4. Ejemplo didáctico de seis vértices

Tomemos $V = \{A, B, C, D, E, F\}$ y el siguiente grafo no dirigido, definido por su lista completa de aristas:

$$E = \{AB, BC, CD, DE, EF, FA, AC, BD, CE, DF\}.$$

El orden de letras en una arista no importa: AB y BA denotan la misma conexión. Toda ruta de la tabla empieza en A y muestra el regreso final a A.

Fuente	Ruta candidata	Enlaces ausentes	F
R_1	$A-B-C-D-F-E-A$	EA	$1/2$
R_2	$A-D-C-B-E-F-A$	AD, BE	$1/3$
R_3	$A-D-B-E-C-F-A$	AD, BE, CF	$1/4$

Paso 1: evaluar y elegir. Las tres rutas visitan los seis vértices una vez antes de volver a A, pero ninguna es todavía válida. Por la ecuación (4.2), la suma de fitness es $1/2 + 1/3 + 1/4 = 13/12$. Sus probabilidades son $p_1 = 6/13$, $p_2 = 4/13$ y $p_3 = 3/13$. Por tanto, una observadora prefiere R_1, aunque podría escoger cualquiera.
Paso 2: intento de empleada. La empleada de R_1 intercambia C y D y obtiene $A-B-D-C-F-E-A$. Faltan CF y EA, así que $b = 2$ y $F = 1/3$. Como empeora respecto de $1/2$, se rechaza y el contador de R_1 aumenta en uno.
Paso 3: intento de observadora. En una realización posible, una observadora selecciona R_1 y luego intercambia F y E. Obtiene $R' = A-B-C-D-E-F-A$. Ahora $b(R') = 0$ y $F(R') = 1$, por lo que se acepta y el contador vuelve a cero.
Paso 4: comprobación independiente. Los vértices antes del regreso son A, B, C, D, E, F, todos distintos y todos los de V. Se revisan las seis aristas: $AB \in E$, $BC \in E$, $CD \in E$, $DE \in E$, $EF \in E$ y, crucialmente, $FA \in E$. Por eso R' sí es un ciclo hamiltoniano. Si faltara FA, sería solo un camino que visita todos los vértices.

Posición	1	2	3	4	5	6
Conexión	$A \to B$	$B \to C$	$C \to D$	$D \to E$	$E \to F$	$F \to A$
¿Existe?	Sí	Sí	Sí	Sí	Sí	Sí

La última columna demuestra el cierre. En una representación sin el A final, esa columna debe generarse de manera explícita al verificar. También se comprueba que la lista contiene seis nombres distintos, no solo que aparecen seis posiciones: una lista como (A, B, C, D, E, E) tendría la longitud correcta, pero repetiría E y omitiría F.
Paso 5: abandono, si no se encontrara una solución. Para ilustrar la fase exploradora por separado, supongamos `limit`$= 2$ y que R_2 prueba dos vecinos sin mejora. Por ejemplo, al intercambiar E y F produce $A-D-C-B-F-E-A$,

con $b = 3$; al intercambiar B y E produce $A-D-C-E-B-F-A$, también con $b = 3$. Ambos se rechazan. El contador llega a dos y se reemplaza R_2 por una permutación nueva. Esta rama es hipotética: si se ejecuta el paso 3 y se encuentra R', el algoritmo ya puede detenerse.

Carácter del ejemplo. Las elecciones de intercambios y la selección de R_1 son una secuencia ilustrativa posible, no una ejecución medida ni un resultado del FHCP.

4.4.1. Qué enseña el ejemplo

El paso de R_1 a R' muestra que una modificación pequeña puede reparar la única arista faltante, pero el primer intento de la empleada también muestra que un intercambio puede empeorar la ruta. La regla voraz evita aceptar esa pérdida en la fuente actual. Las probabilidades no obligan a escoger la mejor fuente: solo hacen que reciba más atención en promedio. Por último, `limit` se mide en intentos fallidos de una fuente, no en la edad total de la colonia. Estas cuatro ideas son las que conviene transmitir al equipo al explicar ABC.

4.5. Aplicación al ciclo hamiltoniano: opciones y dificultades

Opción A: permutaciones completas con penalización. Es la utilizada aquí. Su ventaja es que cada candidato conserva todos los vértices exactamente una vez; se mide cuántas aristas faltan y se busca $b = 0$. Su dificultad es que un solo intercambio puede alterar hasta cuatro enlaces y una meseta de rutas con el mismo b puede estancar la búsqueda.

Opción B: construir caminos válidos de forma incremental. Cada paso añade solo un vecino unido por una arista y no visitado. Así se evitan enlaces inválidos durante la construcción, pero el camino puede quedar bloqueado antes de incluir todos los vértices o sin arista de regreso. Para usar ABC habría que definir con cuidado qué es una fuente, cómo se modifica y cómo se reinicia.

Opción C: operadores o reparación guiados por el grafo. Se pueden preferir intercambios que reparen enlaces ausentes o usar inversión de segmentos y búsqueda local. Es una extensión plausible porque hay adaptaciones combinatorias de ABC para recorridos, pero requiere especificar y evaluar su sesgo y costo computacional.[22]

Dificultades comunes. La calidad parcial no garantiza que un ciclo exista; los óptimos locales y mesetas pueden hacer que la búsqueda repita intentos; las rutas equivalentes por rotación o reverso desperdician evaluaciones; y el grafo puede tener restricciones estructurales que hagan imposible la meta. Un verificador exacto permite confirmar una solución encontrada, pero una búsqueda heurística que se agota no certifica la inexistencia del ciclo. Si la instancia usa aristas dirigidas o un formato concreto, deben ajustarse representación y verificación.

4.5.1. Criterios para interpretar un resultado

Un ciclo producido por el método se puede revisar con un procedimiento determinista: contar vértices, comprobar que no se repiten, consultar cada pareja consecutiva y consultar la arista de cierre. Esa revisión sí confirma un resultado positivo. En cambio, si termina el presupuesto sin ruta válida, la conclusión correcta es que esta ejecución no encontró un ciclo. Puede haber uno en otra región del espacio de búsqueda. La calidad de la mejor ruta encontrada, el número de intentos y el presupuesto utilizado describen el proceso, pero no convierten una búsqueda heurística en una prueba de inexistencia.

Para comparar una futura implementación de ABC con GA de manera justa, ambos tendrían que resolver la misma instancia y usar el mismo verificador de ciclos. El fitness podría compartirse para facilitar la interpretación de resultados; si se usa una función distinta, habría que explicarla. El documento no reporta tiempos, tasas de éxito ni comparaciones, porque esas cifras exigirían ejecuciones reales y un protocolo de medición.

Capítulo 5

Métricas de evaluación y evolución diferencial

5.1. El problema que se quiere resolver

Un grafo $G = (V, E)$ reúne vértices V y aristas E que indican qué pares están conectados. Un ciclo hamiltoniano recorre cada vértice exactamente una vez y vuelve al inicial por una arista existente. Si hay n vértices, una ruta puede escribirse $R = (v_1, v_2, \ldots, v_n)$ y se entiende que después de v_n se intenta regresar a v_1; por eso tiene n conexiones, no $n - 1$. La pregunta de decisión es si existe alguna ruta con esas propiedades, un problema clásico de la teoría de NP-completitud.[23]

Hay que separar camino hamiltoniano y ciclo hamiltoniano. Un camino visita todos los vértices sin repetirlos, pero no necesita regresar al punto de partida. Un ciclo exige además la última arista que une el final con el inicio. Esa condición de cierre es fácil de olvidar cuando la ruta se guarda como una lista, y por eso cualquier comprobación debe tratar la lista como circular.

El problema del viajante (TSP) agrega costos a las aristas y busca el ciclo más barato; aquí la pregunta es de existencia. Aun así, ambos están relacionados: los algoritmos para el TSP pueden adaptarse al problema hamiltoniano.[24] El conjunto FHCP Challenge Set reúne 1001 grafos de 66 a 9528 vértices; se sabe que todos tienen ciclo hamiltoniano y fueron diseñados para resistir a las heurísticas habituales.[24, 25] Su dificultad no viene solo del tamaño, sino de tener pocos ciclos y una estructura que impide descartar aristas con facilidad.[24] En grafos densos, en cambio, suele haber muchos

ciclos y uno aparece pronto.[26] En el concurso asociado, el mejor resultado resolvió 985 grafos y ningún equipo resolvió los 1001,[25] lo que da una idea de la escala del reto. El ejemplo pequeño de la sección 5.5 fue inventado para explicar el método y no pertenece a ese conjunto.

Este trabajo tiene dos partes unidas por una misma preocupación. Como una búsqueda con azar no se comporta igual en cada corrida, primero hay que saber cómo medirla; después se estudia qué método podría usarse, la evolución diferencial, y qué exige adaptarlo.

5.2. Métricas de evaluación

5.2.1. Qué es una métrica y qué se necesita para medir

Una métrica es una regla que convierte el comportamiento de un algoritmo en un número comparable. Antes de calcular cualquiera conviene fijar el vocabulario. Una instancia es un caso concreto del problema, como un grafo. Una ejecución es una corrida completa del algoritmo sobre una instancia; los métodos con azar dan resultados distintos en cada una, y la semilla es el valor que fija esa secuencia de azar para poder repetirla. Una evaluación es calcular una vez la calidad de una solución candidata. El presupuesto es el máximo de recursos permitido a una ejecución.

Una sola corrida no describe un método estocástico: puede haber tenido suerte o mala fortuna. Por eso las medidas se resumen sobre varias ejecuciones independientes.[27] En general interesan tres preguntas distintas, que no conviene mezclar: ¿qué tan seguido lo logra? (fiabilidad), ¿cuánto le cuesta? (esfuerzo) y ¿qué tan cerca queda cuando no lo logra? (calidad). Eiben y Smith separan de forma parecida la velocidad, o eficiencia, de la calidad, o efectividad.[29]

5.2.2. Tiempo, costo computacional, iteraciones y evaluaciones

Las cuatro medidas parecen decir lo mismo, pero miden cosas diferentes.

Medida	Qué mide	Limitación
Tiempo	Tiempo transcurrido o tiempo de procesador (CPU) usado por el programa.	Depende de la máquina, el lenguaje y la implementación.
Costo computacional	Recursos que consume el método: operaciones, tiempo o memoria.	Es un término amplio: hay que declarar qué recurso se cuenta y en qué unidad.
Iteraciones	Ciclos del algoritmo; en métodos evolutivos se llaman generaciones.	Una generación cuesta distinto según el tamaño de la población.
Evaluaciones	Veces que se calculó la calidad de una solución.	No todas cuestan lo mismo.

Las evaluaciones son la medida independiente de la máquina más común en optimización continua, y la plataforma COCO las usa precisamente porque el tiempo de CPU depende del lenguaje, el estilo de programación y el equipo.[27, 28] La recomendación práctica es reportar evaluaciones y tiempo de CPU, y si se dan generaciones, incluir también las evaluaciones.[27] La advertencia es similar en Eiben y Smith: no conviene contar generaciones cuando se comparan configuraciones con poblaciones distintas.[29]

Ejemplo de la diferencia. En evolución diferencial cada generación evalúa una candidata por cada miembro de la población, además de la evaluación inicial. Con 20 individuos y 50 generaciones hay $20 + 20 \cdot 50 = 1020$ evaluaciones; con 40 individuos y las mismas 50 generaciones hay 2040. Decir "50 generaciones" oculta que el segundo caso costó el doble.

Propuesta para este problema: cada evaluación de una ruta revisa n conexiones, de modo que evaluar en un grafo de 3000 vértices cuesta mucho más que en uno de 66. Por eso las evaluaciones solo se comparan directamente entre instancias de tamaño parecido, y el tiempo debe acompañarlas cuando el método añade trabajo interno.

5.2.3. Tasa de éxito y ejecuciones fallidas

Hablar de éxito exige definir un objetivo. La tasa de éxito es el porcentaje de ejecuciones que lo alcanzan dentro del presupuesto. A partir de ahí hay dos formas de plantear una medición: con objetivo fijo se pregunta cuánto esfuerzo costó llegar, y con presupuesto fijo se pregunta a qué calidad se llegó con ese esfuerzo. Hansen y colaboradores prefieren la primera porque da resultados cuantitativos e interpretables.[28]

El punto delicado son los fallos. En una ejecución fallida el tiempo hasta el éxito no está definido: solo se sabe cuánto consumió.[28] Quien mide con objetivo fijo debe declarar cómo los trata.[27] Si simplemente se eliminan, el método parece mejor de lo que fue. Las medidas habituales que sí los incluyen son:

- aRT (average runtime, también llamado ERT): suma las evaluaciones de todas las ejecuciones y divide entre el número de éxitos. Estima cuánto costaría llegar a un éxito si se reiniciara el algoritmo cada vez que falla.[27, 28]
- PAR-k: promedia lo consumido contando cada fallo como k veces el presupuesto máximo; PAR2 y PAR10 son los casos comunes.[27]
- Mediana con fallos al final: los fallos se ordenan por encima de todo éxito. Sigue siendo válida mientras al menos la mitad de las ejecuciones tenga éxito, y los cuantiles son más robustos que la media.[27]

Ejemplo con diez ejecuciones inventadas. Con un presupuesto de 1000 evaluaciones, siete ejecuciones tuvieron éxito con 120, 150, 180, 200, 240, 300 y 410 evaluaciones, y tres fallaron.

Resumen	Cálculo	Valor	Qué muestra
Tasa de éxito	7/10	70 %	Fiabilidad.
Media solo de éxitos	1600/7	228.6	Oculta los tres fallos.
Mediana con fallos al final	entre 240 y 300	270	Una ejecución típica.
aRT	$(1600 + 3000)/7$	657.1	Esfuerzo total por cada éxito.
PAR10	$(1600 + 30\,000)/10$	3160	Castiga con fuerza los fallos.

La misma información da 228.6 o 657.1 según cómo se trate a los fallos, y por eso el informe debe decir qué medida usó. Además, una tasa calculada con pocas ejecuciones es incierta. Para muestras pequeñas se recomiendan los intervalos de Wilson o Jeffreys, porque el intervalo clásico de Wald tiene una cobertura errática.[30] En este ejemplo, 7/10 da un intervalo de Wilson al 95 % de aproximadamente 40 % a 89 %: el "70 %" es mucho menos exacto de lo que aparenta.

Propuesta: registrar la causa de cada fallo (presupuesto agotado, límite de tiempo o error técnico), porque un error del programa no informa sobre el método. Y si ninguna ejecución tiene éxito, el aRT no está definido, así que debe reportarse "0 de N" junto con el presupuesto usado.

5.2.4. Precisión, recall y F1

Estas medidas vienen de la clasificación, donde cada elemento se etiqueta como positivo o negativo. Un verdadero positivo (TP) se señaló como positivo y lo era; un falso positivo (FP) se señaló sin serlo; un falso negativo (FN) era positivo y no se señaló. Con esos conteos:[31]

$$\text{Precisión} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2\,TP}{2\,TP + FP + FN}. \tag{5.1}$$

La precisión responde: de lo que señalé, ¿cuánto era correcto? El recall o exhaustividad responde: de lo que debía encontrar, ¿cuánto encontré? F1 es su media armónica. Ninguna usa los verdaderos negativos, y pares distintos de precisión y recall pueden dar el mismo F1, por lo que no debe reportarse solo.[31]

Ejemplo de clasificación. De 12 mensajes de spam, un filtro marcó 8 correctamente y además marcó 2 mensajes normales. Entonces $TP = 8$, $FP = 2$ y $FN = 4$. La precisión es $8/10 = 0.80$, el recall es $8/12 \approx 0.67$ y F1 es aproximadamente 0.73.

Lo importante es que estas fórmulas no significan nada hasta responder cuatro preguntas: (1) qué elemento se clasifica, (2) qué significa positivo, (3) cuál es la referencia que dice qué es correcto y (4) en qué nivel se mide: conexión, ruta, ejecución o instancia. En el spam las respuestas son obvias; en el ciclo hamiltoniano no.

5.2.5. Por qué no basta con un porcentaje de conexiones válidas

Las fuentes revisadas no ofrecen una definición estándar de estas medidas para el ciclo hamiltoniano. La siguiente tabla es una propuesta que aplica la ecuación (5.1) al grafo de la sección 5.5, que tiene 9 aristas. M es la ruta inválida (A, D, C, B, F, E), a la que solo le falta la conexión de regreso EA. C_3 es un ciclo perfecto y C_1 es otro ciclo del mismo grafo.

Qué se toma como positivo	Caso	P / R / F1	Problema
La conexión existe en el grafo	M	.83 / .56 / .67	La precisión es el porcentaje de conexiones válidas.
	C_3	1 / .67 / .80	Un ciclo perfecto nunca llega a recall 1: usa 6 de las 9 aristas.
La conexión pertenece a un ciclo de referencia (C_1)	M	.50 / .50 / .50	Las tres medidas coinciden.
	C_3	.50 / .50 / .50	Un ciclo perfecto empata con una ruta inválida.
La ejecución entrega un ciclo verificado	10 ejec.	1 / .70 / .82	Solo repite la tasa de éxito.

Con un ciclo de referencia, el resultado no es casualidad. La ruta y la referencia tienen n conexiones cada una, así que $TP + FP = TP + FN = n$, y por tanto precisión, recall y F1 valen siempre TP/n. Son tres nombres para un solo porcentaje, justo lo que el enunciado advierte que no basta. Peor aún, si el grafo tiene varios ciclos, la medida castiga una solución correcta solo por ser distinta de la elegida como referencia.

A nivel de ejecución tampoco se gana información. Si el éxito se declara después de verificar el ciclo, no hay falsos positivos y la precisión vale 1; el recall coincide con la tasa de éxito y $F_1 = 2R/(1 + R)$ es solo una transformación suya. Además, como todos los grafos del FHCP tienen ciclo,[25] no existe clase negativa a nivel de instancia. Por eso esta investigación no adopta una definición: la deja planteada como duda para el profesor.

5.2.6. Qué medidas parecen aplicables

La siguiente valoración es una propuesta pendiente de confirmar.

Medida	Significado en el ciclo hamiltoniano	Valoración
Tasa de éxito con intervalo	Porcentaje de ejecuciones con ciclo verificado.	Aplicable.
Evaluaciones y tiempo	Esfuerzo hasta el éxito o hasta agotar el presupuesto.	Aplicable.
aRT, PAR-k, mediana	Resumen de esfuerzo que incluye los fallos.	Transferible; confirmar.
Conexiones faltantes	Las que no existen en la mejor ruta final.	Calidad parcial.
Porcentaje de conexiones válidas	Avance parcial; no significa haber resuelto.	Con su propio nombre.
Precisión, recall y F1	Dependen de la definición elegida.	Duda abierta.

5.3. Evolución diferencial: el algoritmo original

5.3.1. Origen e idea central

La evolución diferencial (DE, Differential Evolution) fue propuesta por Storn y Price para optimizar funciones sobre valores continuos.[32] Sigue siendo una de las familias de optimizadores evolutivos más estudiadas, con una extensa literatura sobre variantes y parámetros.[33] Mantiene una población de NP soluciones candidatas, cada una representada como un vector de D números reales, y una función objetivo les asigna un costo que se quiere minimizar.

Su idea distintiva es simple: en lugar de usar una distribución de azar externa para decidir cuánto moverse, usa la diferencia entre dos soluciones de la propia población. Cuando la población está dispersa, las diferencias son grandes y los saltos también; cuando se concentra, los saltos se acortan solos. Frente al algoritmo genético, donde se seleccionan padres y se forman descendientes, en DE cada candidata compite directamente contra el vector al que quiere reemplazar.[32]

5.3.2. Qué ocurre en una generación

La variante básica se llama DE/rand/1/bin: el vector base se elige al azar (rand), se usa una diferencia (1) y el cruce es binomial (bin).[32] En cada generación, cada vector objetivo x_i pasa por tres pasos:

1. Mutación. Se eligen tres vectores distintos entre sí y distintos de x_i, y se

forma el vector mutante

$$v = x_{r_1} + F\,(x_{r_2} - x_{r_3}), \tag{5.2}$$

donde F es el factor de escala. Por eso se requieren al menos cuatro vectores.[32]

2. Cruce. Se forma el vector de prueba u. Cada componente viene de v con probabilidad CR y de x_i en caso contrario, excepto una posición elegida al azar que siempre viene de v; así se garantiza tomar al menos una componente del donante, aunque el valor o la permutación decodificada pueden coincidir con los anteriores.[32]

3. Selección. Se evalúa u. Si cuesta menos que x_i, lo reemplaza en la siguiente generación; si no, x_i se conserva.[32]

Ejemplo numérico. Se quiere minimizar $f(x, y) = x^2 + y^2$ con $x_1 = (3, 1)$, $x_2 = (0, 3)$, $x_3 = (1, -1)$, $x_4 = (-2, 2)$, $F = 0.5$ y $CR = 0.5$. Para el objetivo x_1 se eligen x_3 como base y $x_2 - x_4 = (2, 1)$ como diferencia, así que $v = (1, -1) + 0.5\,(2, 1) = (2, -0.5)$. En el cruce, la posición 2 es la forzada; en la posición 1 el azar da 0.7, mayor que CR, y se conserva el 3 de x_1. Queda $u = (3, -0.5)$, con costo 9.25, menor que el costo 10 de x_1. Por eso u ocupa su lugar en la siguiente generación.

5.3.3. Parámetros que influyen

Storn y Price sugieren NP entre $5D$ y $10D$, nunca menos de 4; $F = 0.5$ como punto de partida, con valores por debajo de 0.4 o por encima de 1 eficaces solo en ocasiones; y $CR = 0.1$ como primera opción, probando primero 0.9 o 1 para ver si se obtiene una solución rápida. Si la población converge prematuramente, aconsejan aumentar F o NP.[32] La tabla siguiente resume el efecto esperado a partir de esas indicaciones y de la ecuación (5.2).

Ajuste	Si se reduce	Si se aumenta
NP	Menos diversidad; cada generación cuesta menos.	Más diversidad y más evaluaciones por generación.
F	Saltos cortos; riesgo de convergencia prematura.	Saltos largos; más exploración y búsqueda menos fina.
CR	El vector de prueba se parece más al objetivo.	Hereda más del mutante; a menudo acelera la convergencia.

Propuesta: esas recomendaciones son de 1997 y para problemas continuos, así que aquí solo sirven como punto de partida. Aplicadas a una ruta de 3000 vértices darían entre 15 000 y 30 000 vectores, algo que habría que justificar. La adaptación automática de parámetros es un tema desarrollado después.[33]

5.3.4. Cuándo termina la búsqueda

Los criterios de paro para DE se agrupan en cinco clases: por referencia (se alcanza un valor), por agotamiento (se acaba el presupuesto), por mejora (deja de mejorar), por movimiento (la población deja de moverse) y por distribución (la población se concentra).[34] En ese estudio, hecho sobre un problema real concreto, algunos criterios de mejora resultaron poco fiables, y los autores advierten que los resultados pueden cambiar en otros casos.[34]

5.4. Adaptación a órdenes de visita

5.4.1. Por qué no se puede aplicar directamente

DE necesita sumar, restar y escalar vectores, y una permutación, un orden de todos los vértices sin repetir, no admite esas operaciones con sentido. Si se numeran los vértices, $(1, 2, 3, 4, 5)$ y $(2, 3, 4, 5, 1)$ son el mismo ciclo con otro inicio, pero su resta es $(-1, -1, -1, -1, 4)$. Aplicar la mitad de esa diferencia a $(1, 3, 2, 4, 5)$ produce (0.5, 2.5, 1.5, 3.5, 7), que no es una ruta: los valores ni siquiera son vértices. La ecuación (5.2) supone que estar "cerca" en los números significa parecerse como solución, y eso aquí no ocurre.

Hay además una redundancia propia del problema. En un grafo no dirigido, cada ciclo se escribe de $2n$ formas: n puntos de inicio por 2 sentidos. Con 5 vértices, las 120 permutaciones representan solo 12 ciclos candidatos distintos. Fijar el vértice inicial elimina una parte de esa redundancia, y la del sentido queda.

5.4.2. Función de evaluación y validez

Sea $R = (v_1, \ldots, v_n)$ una permutación y $v_{n+1} = v_1$. Una evaluación simple, propuesta para este documento, cuenta las conexiones inexistentes:

$$b(R) = \sum_{i=1}^{n} \mathbf{1}\big[(v_i, v_{i+1}) \notin E\big]. \tag{5.3}$$

La expresión $\mathbf{1}[P]$ vale 1 si P es verdadera y 0 si no, de modo que $b(R)$ revisa exactamente n conexiones, incluida la de regreso. Como DE minimiza, b puede usarse directamente como costo: no hace falta transformarlo en un fitness que se maximice. En este documento la letra F se reserva para el factor de escala de DE; conviene no confundirla con la $F(R)$ que otros métodos del equipo usen como fitness.

La validez se comprueba aparte: (1) la ruta contiene exactamente los n vértices distintos; (2) cada par consecutivo es una arista; y (3) también existe la arista del último al primero. Si la representación garantiza una permutación, $b(R) = 0$ equivale a validez, pero conviene conservar un verificador independiente para detectar errores. Una ruta con $b = 1$ puede estar cerca según esta medida, pero nunca es una solución.

5.4.3. Pseudocódigo de la adaptación

El esquema siguiente es deliberadamente general: conserva las fases de DE y deja indicados los puntos que cambian según la opción de la sección 5.6. No es un programa completo ni supone un lenguaje.

1. Recibir el grafo $G = (V, E)$, los parámetros NP, F y CR, el presupuesto de evaluaciones B y una representación de las rutas.

2. Crear NP soluciones iniciales al azar, obtener su ruta y calcular b con la ecuación (5.3). Poner el contador de evaluaciones en NP y guardar la mejor.

3. Si alguna tiene $b = 0$, pasar al paso 10. Exigir $B \geq NP$ y pasar también al paso 10 si se agotó el presupuesto tras inicializar. Exigir $B \geq NP$ y pasar también al paso 10 si se agotó el presupuesto tras inicializar.

4. Para cada solución x_i, elegir tres soluciones distintas entre sí y distintas de x_i.

5. Mutar con ellas para obtener v. Ajuste A: con vectores es la ecuación (5.2); con permutaciones hay que definir la diferencia y la escala.

6. Cruzar x_i y v con probabilidad CR para obtener u. Ajuste B: con permutaciones debe evitar vértices repetidos.

7. Obtener la ruta de u. Ajuste C: no existe en DE original; con claves consiste en ordenarlas.

8. Calcular b y sumar uno al contador. Seleccionar: si la candidata mejora a x_i, ocupa su lugar en la siguiente generación; si no, x_i se conserva. Ajuste D: decidir si un empate también se acepta. Actualizar la mejor ruta y, si $b = 0$ o se agotó B, pasar al paso 10.

9. Al terminar la población, reemplazarla por la nueva y repetir desde el paso 4.

10. Comprobar la mejor ruta con el verificador independiente. Si es válida, devolverla junto con las evaluaciones usadas. Si no, informar "no encontrado con este presupuesto de búsqueda".

Propuesta de paro: aquí existe un criterio de referencia natural, $b = 0$, porque esa ruta se comprueba como ciclo. Debe combinarse con un presupuesto para que las ejecuciones fallidas también terminen. Reiniciar tras un estancamiento es otra posibilidad por discutir.

5.5. Ejemplo didáctico de seis vértices

Tomemos $V = \{A, B, C, D, E, F\}$ y el siguiente grafo no dirigido, definido por su lista completa de aristas:

$$E = \{AB, BC, CD, DE, EF, FA, AD, BF, CE\}.$$

Tiene tres ciclos hamiltonianos: C_1, *A-B-C-D-E-F-A*; C_2, *A-B-F-E-C-D-A*; y C_3, *A-D-E-C-B-F-A*. El ejemplo usa claves aleatorias, descritas en la sección 5.6, solo porque hacen visible la aritmética de DE; no significa que se hayan elegido como adaptación. Cada vector tiene una clave por vértice en el orden A, B, C, D, E, F. Para obtener la ruta, las claves se ordenan de menor a mayor y se leen los vértices correspondientes. Se usa $NP = 4$, $F = 0.5$ y $CR = 0.5$.

Vector	Claves	Ruta (con regreso)	Ausentes	b
x_1	(.3, .7, .1, .8, .9, .6)	*C-A-F-B-D-E-C*	CA, BD	2
x_2	(.4, .2, .9, .7, .6, .8)	*B-A-E-D-F-C-B*	AE, DF, FC	3
x_3	(.4, .3, .9, .7, .5, .2)	*F-B-A-E-D-C-F*	AE, CF	2
x_4	(.9, .2, .7, .3, .6, .5)	*B-D-F-E-C-A-B*	BD, DF, CA	3

Paso 1: evaluar. Las cuatro rutas visitan los seis vértices, pero ninguna es válida. En x_1, la clave menor es 0.1 y pertenece a C, por lo que la ruta empieza en C. Evaluar la población costó 4 evaluaciones.
Paso 2: primer intento, rechazado. Para el objetivo x_1 se usa $v = x_2 + 0.5\,(x_3 - x_4) = (.15,\ .25,\ 1.0,\ .9,\ .55,\ .65)$. Los azares por posición son .2, .8, .4, .9, .6 y .3, y la posición forzada es la 5. Se toman de v las posiciones 1, 3, 5 y 6, lo que da $u = (.15,\ .7,\ 1.0,\ .8,\ .55,\ .65)$. Su ruta es *A-E-F-B-D-C-A*, y faltan AE, BD y CA, así que $b = 3$. Como 3 no es menor que 2, se rechaza y x_1 se conserva. Van 5 evaluaciones.
Paso 3: segundo intento, aceptado. Para el objetivo x_2 se usa $v = x_3 + 0.5\,(x_4 - x_1) = (.7,\ .05,\ 1.2,\ .45,\ .35,\ .15)$. Los azares son .7, .1, .9, .3, .6 y .8, y la posición forzada es la 6. Se toman de v las posiciones 2, 4 y 6, lo que da $u = (.4,\ .05,\ .9,\ .45,\ .6,\ .15)$. Su ruta es $R' = B\text{-}F\text{-}A\text{-}D\text{-}E\text{-}C\text{-}B$ con $b = 0$. Como $0 < 3$, reemplaza a x_2 y la búsqueda se detiene con 6 evaluaciones.
Paso 4: comprobación independiente. Los vértices antes del regreso son B, F, A, D, E, C, todos distintos y todos los de V. La última columna de la tabla corresponde a la conexión de cierre. R' es el ciclo C_3 escrito desde otro inicio.

Posición	1	2	3	4	5	6
Conexión	$B \to F$	$F \to A$	$A \to D$	$D \to E$	$E \to C$	$C \to B$
¿Existe?	Sí	Sí	Sí	Sí	Sí	Sí

Contraejemplo del cierre. La ruta $M = (A, D, C, B, F, E)$ usa cinco conexiones existentes: AD, DC, CB, BF y FE. Solo falla al regresar, porque EA no es arista, así que $b = 1$. Visita todos los vértices y aun así no es un ciclo: es un camino hamiltoniano.

Carácter del ejemplo. Las claves y los números al azar se escogieron para que el ejemplo fuera breve. Es una secuencia ilustrativa posible, no una ejecución medida ni un resultado del FHCP; hallar un ciclo en dos intentos no es lo esperable.

5.5.1. Qué enseña el ejemplo

El primer intento muestra que una candidata puede empeorar, y que la selección impide perder lo ganado. El segundo muestra que cambiar pocas claves puede reordenar toda la ruta: aunque u conserva tres claves de x_2, su ruta solo conserva dos de las seis conexiones. Esa es una virtud para explorar y una dificultad para mejorar con precisión. Por último, x_1 y x_3 tenían $b = 2$ y empataban sin parecerse como rutas, lo que anticipa el problema de las mesetas.

5.6. Aplicación al ciclo hamiltoniano: opciones y dificultades

Esta investigación no elige una opción. La decisión depende de las dudas de la sección 5.7 y de pruebas que aquí no se realizaron.

Opción A: claves aleatorias. Se conserva DE continua y cada vector se ordena para leer una ruta; la técnica procede de los algoritmos genéticos[35] y se ha estudiado en DE para permutaciones, incluido el problema del viajante.[36] La regla del valor de posición menor (SPV) sigue una idea relacionada.[38, 39] Su ventaja es que toda clave da una permutación válida y DE no cambia. Su dificultad es que infinitos vectores representan la misma ruta, lo que crea mesetas en la búsqueda; Santucci y colaboradores señalan esto como una causa probable de su bajo rendimiento.[37]

Opción B: DE algebraica. Se trabaja directamente con permutaciones y se redefinen las operaciones: la diferencia entre dos rutas es una secuencia de intercambios adyacentes, y escalarla por F significa aplicar solo una parte de ella.[37] Su ventaja es que no hay decodificación ni distinción entre vector y ruta. Su dificultad es que es más compleja y fue probada en secuenciación de tareas, no en este problema.

Opción C: otros enfoques discretos. El manual de Onwubolu y Davendra reúne enfoques como la transformación directa e inversa y los índices de posición relativa.[38] Queda pendiente revisarlos, porque solo se consultó su índice. Santucci y colaboradores también mencionan una DE geométrica aplicada al TSP con resultados pobres.[37]

Vacío detectado. Una búsqueda dirigida no localizó un artículo que aplique DE específicamente al ciclo hamiltoniano. Lo más cercano es el TSP.[36] Esto no prueba que no exista, pero sí significa que todo lo anterior es trans-

ferencia desde problemas parecidos.

Dificultades comunes. Se trata de observaciones propuestas por este trabajo:

- Mesetas. b solo toma valores enteros, por lo que muchas rutas empatan. Con selección estricta, una candidata igual se descarta y la población puede quedar inmóvil.
- Grafos dispersos. Una ruta al azar tiene en promedio $2m/(n-1)$ conexiones válidas, donde m es el número de aristas. En un grafo de 3000 vértices y grado 3 serían unas 3 de 3000, así que la búsqueda arranca casi sin información útil.
- Redundancia. Dos escrituras del mismo ciclo pueden estar lejos como vectores, lo que debilita el sentido de la diferencia.
- Calidad engañosa. Una ruta con una conexión faltante no necesariamente está cerca de un ciclo; quizá ningún cambio pequeño la repare.

5.6.1. Criterios para interpretar un resultado

Un ciclo producido por el método se revisa con un procedimiento determinista: contar vértices, comprobar que no se repiten, consultar cada pareja consecutiva y consultar la arista de cierre. Esa revisión sí confirma un resultado positivo. En cambio, si se agota el presupuesto, la conclusión correcta es que esta ejecución no encontró un ciclo. En el FHCP se sabe además que el ciclo existe,[25] de modo que un fallo informa sobre el método y no sobre el grafo.

Para comparar una futura implementación con otros métodos del equipo de manera justa, todos tendrían que resolver las mismas instancias, usar el mismo verificador y contar el esfuerzo igual. También convendría guardar la semilla de cada ejecución y usar los mismos puntos de partida al comparar.[27] Este documento no reporta tiempos ni tasas de éxito reales, porque esas cifras exigirían ejecuciones y un protocolo de medición; todos los números de sus ejemplos son didácticos.

5.7. Dudas para el profesor

1. ¿Precisión, recall y F1 son obligatorios o basta justificar por qué no aplican? Si son obligatorios, ¿qué definición de positivo y qué referencia se aceptan?

2. Si existen varios ciclos válidos, ¿cómo debe tratarse la referencia?

3. ¿El éxito es únicamente un ciclo completo, o también se valoran rutas parciales?

4. ¿Cuántas ejecuciones por instancia, qué presupuesto y qué regla para fallos se esperan?

5. ¿Qué debe reportarse como costo computacional: evaluaciones, tiempo o ambos?

6. ¿Se añadirán grafos dirigidos fuera del conjunto FHCP? La integración considera grafos simples no dirigidos para este benchmark.

7. ¿Se permite combinar DE con búsqueda local, o debe estudiarse sola?

8. ¿Qué instancias y tamaños del FHCP se usarán, si es el caso?

Capítulo 6

FHCP Challenge y enjambre de partículas

6.1. Métricas del FHCP Challenge

El Flinders Hamiltonian Cycle Project (FHCP) define el HCP como el problema de determinar si un grafo de N vértices contiene un ciclo simple de longitud N. [24]

6.1.1. Características del benchmark

- El FHCP Challenge Set contiene 1,001 instancias del HCP. [9]
- Los tamaños de los grafos van desde 66 hasta 9,528 vértices. [9]
- El tamaño promedio es ligeramente superior a 3,000 vértices. [9]
- Las instancias fueron diseñadas para ser difíciles debido a su estructura, no únicamente por su tamaño. [24]
- Todas las instancias tienen al menos un ciclo hamiltoniano conocido. [24]

6.1.2. Formato de los archivos

Los archivos del FHCP utilizan el formato HCP, estructurado en tres secciones:

- Encabezado.
- Lista de aristas.
- Pie de archivo.

El campo `DIMENSION` indica el número de vértices. El campo `EDGE_DATA_FORMAT : EDGE_LIST` indica que las conexiones se representan mediante una lista de aristas. [9]

Los archivos de solución o tour contienen la secuencia de vértices en el orden en que son visitados por el ciclo hamiltoniano. [9]

6.1.3. Criterios de comparación

Para realizar una comparación justa entre algoritmos, se deben fijar:

- El mismo subconjunto de instancias.
- El mismo límite de tiempo o número de evaluaciones.
- Varias ejecuciones independientes por instancia.
- La misma definición de éxito.
- La verificación de que el ciclo encontrado sea válido.
- Métricas de tiempo, evaluaciones, tasa de éxito y desviación estándar.

6.1.4. Reglas históricas y alcance de la comparación

El concurso histórico estuvo abierto del 30 de septiembre de 2015 al 30 de septiembre de 2016. Ganaba la entrega con más grafos resueltos; en caso de empate se consideraba la recibida primero. El mejor equipo resolvió 985 instancias. Las reglas exigían soluciones en el formato correcto. Esta cobertura es una referencia histórica y no un tiempo de ejecución comparable directamente con nuestro equipo. Las semillas, repeticiones y presupuestos uniformes antes propuestos son un protocolo académico adicional, no exigencias oficiales del concurso.[9]

6.1.5. Ejemplo completo de entrada y salida

Este ejemplo propio contiene cinco vértices. Cada par de la entrada representa una arista no dirigida. Los terminadores -1 y EOF no son vértices.

```
NAME : ciclo5
COMMENT : Ejemplo didactico
TYPE : HCP
DIMENSION : 5
EDGE_DATA_FORMAT : EDGE_LIST
EDGE_DATA_SECTION
1 2
2 3
3 4
4 5
5 1
-1
EOF
```

La solución lista cada vértice una sola vez; el cierre es implícito:

```
NAME : ciclo5_solucion
COMMENT : Ciclo verificado
TYPE : TOUR
DIMENSION : 5
TOUR_SECTION
1
2
3
4
5
-1
EOF
```

Se propone validar rangos de identificadores, encabezados y aristas; conservar la numeración de entrada y restaurarla al exportar. El verificador debe comprobar cobertura, unicidad, adyacencias y cierre, no igualdad exacta con un tour de referencia: un grafo puede tener varios ciclos correctos.[9]

6.2. Particle Swarm Optimization (PSO)

PSO modela un enjambre de partículas que ajustan su posición según su mejor experiencia personal y la mejor experiencia colectiva.

La actualización de velocidad se define como:

$$v_i^{t+1} = wv_i^t + c_1 r_1 (p_i - x_i^t) + c_2 r_2 (g - x_i^t)$$

La actualización de posición es:

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

donde:

- w es el coeficiente de inercia.
- c_1 y c_2 son coeficientes de aprendizaje.
- r_1 y r_2 son valores aleatorios.
- p_i es la mejor posición personal.
- g es la mejor posición global.

Para el HCP, PSO requiere una codificación discreta, como random keys, o una variante discreta con operadores definidos sobre intercambios. Las ecuaciones continuas no pueden aplicarse directamente a los identificadores de vértices. [47, 48]

6.2.1. Alcance de las ecuaciones

La formulación con peso de inercia w corresponde a la modificación de Shi y Eberhart de 1998; no se atribuye como tal al artículo original de 1995. Los factores aleatorios se generan independientemente por componente y actualización; los productos con vectores se interpretan componente a componente. p_i guarda la posición con mejor costo personal y g la mejor conocida del enjambre en una topología global.[47, 48]

6.2.2. Decodificación y pseudocódigo propuestos

Se complementa la propuesta de investigacion.tex con claves aleatorias: una coordenada real por vértice y un ordenamiento para obtener la permutación. Claves $(0.4, 0.1, 0.8, 0.3, 0.6)$ para (A, B, C, D, E) producen (B, D, A, E, C). Un empate se resuelve por identificador. La representación garantiza unicidad y cobertura, pero no la existencia de las conexiones. La técnica de claves procede de Bean; usarla con PSO y el costo de conexiones ausentes es aquí una propuesta de adaptación.[35]

1. Recibir el grafo, tamaño del enjambre S, parámetros w, c_1, c_2, semilla y presupuesto. Definir límites: por ejemplo, recortar claves a $[0, 1]$ y velocidades a $[-v_{\text{máx}}, v_{\text{máx}}]$.

2. Inicializar posiciones y velocidades, ordenar claves y evaluar el costo b de cada permutación. Contar también estas evaluaciones. Guardar los mejores personales y global y verificar cualquier costo cero.

3. Mientras queden recursos, actualizar velocidad y posición de cada partícula, y aplicar los límites establecidos.

4. Decodificar, evaluar y contabilizar el esfuerzo. Actualizar el mejor personal si el costo disminuye; conservar el anterior en empate.

5. Actualizar el mejor global. En esta propuesta se hace después de cada partícula; una variante sincrónica lo haría al final de la iteración.

6. Verificar cualquier candidata de costo cero y devolverla si es válida. Si se agotan los recursos, devolver la mejor candidata con el estado "no encontrado", sin afirmar inexistencia.

6.2.3. Ejemplo didáctico y dificultades

Sobre $E = \{AB, BC, CD, DE, EA\}$, las claves $(0.1, 0.2, 0.4, 0.3, 0.5)$ producen (A, B, D, C, E), con dos conexiones ausentes. Un desplazamiento $(0, 0, -0.1, 0.1, 0)$ produce claves ordenadas y la ruta (A, B, C, D, E): se verifican AB, BC, CD, DE, EA. Ese desplazamiento puede corresponder a una velocidad previa igual, con $w = 1$ y términos de atracción nulos en ese paso. Es una ilustración algebraica elegida a mano, no una ejecución ni una recomendación de parámetros.

La decodificación cuesta $O(n \log n)$ con ordenamiento por comparación; verificar conexiones cuesta $O(n)$ con consultas constantes de adyacencia. Almacenar el enjambre y sus recuerdos requiere $O(Sn)$ espacio además del grafo. Diferentes vectores pueden decodificar a la misma permutación, produciendo mesetas. La convergencia del enjambre puede reducir diversidad sin alcanzar un ciclo. Reinicios, límites y topología social son decisiones que deben documentarse y medirse, sin prometer éxito en un presupuesto finito.

Capítulo 7

Síntesis de diseño para la fase 1

7.1. Propuesta de diseño experimental

Para implementar y comparar los algoritmos, se recomienda seguir los siguientes pasos:

1. Definir el HCP como un problema de decisión sobre grafos no dirigidos.
2. Utilizar permutaciones cíclicas como representación principal.
3. Validar que cada candidata tenga una estructura de permutación antes de evaluar sus conexiones; permitir conexiones ausentes como penalizaciones durante la búsqueda.
4. Para permutaciones completas, contar conexiones ausentes, incluido el cierre. Si se usan otras codificaciones, definir también restricciones de cobertura y conectividad.
5. Implementar SA, ACO, ABC, DE y PSO bajo la misma interfaz de evaluación.
6. Utilizar el FHCP Challenge Set como benchmark.
7. Medir tasa de éxito, tiempo, número de evaluaciones, iteraciones y desviación estándar.
8. Verificar cada ciclo encontrado antes de reportarlo como solución válida.
9. Comparar los algoritmos bajo el mismo presupuesto computacional.

La fortaleza del diseño está en separar tres capas:

- Representación.
- Búsqueda.
- Evaluación.

Esto permite comparar los cinco algoritmos bajo condiciones equivalentes y obtener resultados reproducibles sobre el FHCP Challenge Set. [24]

7.2. Precisiones de la integración

Se adopta la separación entre representación, búsqueda y evaluación propuesta en investigacion.tex. El decodificador transforma el estado interno en una ruta; el fitness orienta la búsqueda; un verificador independiente decide si la ruta satisface exactamente la definición de ciclo.

OX y PMX son cruces especializados para permutaciones. Intercambio, inserción e inversión conservan los vértices, pero pueden romper conexiones. Si se codifican aristas, exigir grado 2 en cada vértice no basta: pueden aparecer varios ciclos disjuntos; también debe exigirse conectividad.[4]

Se corrige el objetivo combinado de violaciones y longitud sugerido en investigacion.tex: toda permutación circular tiene n conexiones candidatas, así que su longitud combinatoria no discrimina calidad. El costo de conexiones ausentes sí distingue incumplimientos. Una lista con duplicados no debe evaluarse como si fuera una permutación válida.

ACO puede construir caminos parciales; su calidad interna no equivale al costo de una permutación completa. La comparación entre algoritmos usa el mismo éxito verificado, y cualquier calidad parcial declara si la ruta es completa o parcial.

7.3. Precisión, recall y F1: límites de aplicación

Para clasificación con referencia independiente,

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}.$$

Si un denominador es cero, se informa que la métrica no está definida, salvo que se declare una convención de reporte.[31]

Se matiza investigacion.tex: agotar el tiempo sin encontrar ciclo es un resultado inconcluso, no una prueba de que el grafo sea negativo. FHCP contiene solo positivos. Si "detección" significa encontrar un ciclo verificado, el recall coincide con la tasa de detección, la precisión vale 1 cuando hay detecciones y F1 es una transformación del recall. No son tres pruebas independientes de rendimiento. Evaluar un clasificador de hamiltonicidad exigiría también negativos certificados y una política para resultados inconclusos. Comparar aristas contra un único tour penalizaría otros ciclos correctos.

7.4. Registro y reproducibilidad

Se propone elegir las instancias antes de experimentar y separar grafos de calibración de los de evaluación. Registrar por ejecución: identificador, n, m, algoritmo, versión, parámetros, semilla, presupuesto, evaluaciones, iteraciones, tiempo, mejor costo y estado final. Guardar cada ciclo como TOUR y verificarlo independientemente.

Deben documentarse equipo y entorno, distinguir tiempo de CPU del tiempo de pared e incluir el costo de decodificación, reparación, búsqueda local y reinicios. Igualar generaciones no iguala esfuerzo: una generación poblacional puede evaluar muchas candidatas. Las evaluaciones deben acompañarse de tiempo, pues las operaciones internas difieren entre métodos.[27, 28]

Reportar tasa de éxito por instancia y cobertura del conjunto por separado; añadir incertidumbre y dispersión. No descartar fallos al calcular promedios sin explicarlo. El esfuerzo total dividido entre el número de éxitos estima esfuerzo hasta éxito bajo reinicios; con cero éxitos no produce una estimación finita. Las propuestas de benchmarking se transfieren desde optimización y no se presentan como reglas oficiales del FHCP.[27, 28]

7.5. Decisiones pendientes

Antes de implementar se acuerdan instancias, presupuestos, repeticiones, calibración y uso de búsqueda local. El texto conserva GA como fundamento del módulo evolutivo; los cinco algoritmos solicitados son SA, ACO, ABC, DE y PSO. Todos los ejemplos son didácticos y no prueban superioridad de

un método.

Bibliografía

[1] M. Dorigo, V. Maniezzo y A. Colorni. «Ant System: Optimization by a colony of cooperating agents». IEEE Trans. Systems, Man, and Cybernetics, Part B, 26(1):29–41, 1996. https://doi.org/10.1109/3477.484436
Respalda: regla de transición probabilística, evaporación y depósito del Ant System.

[2] M. Dorigo y L. M. Gambardella. «Ant Colony System: A cooperative learning approach to the traveling salesman problem». IEEE Trans. Evolutionary Computation, 1(1):53–66, 1997. https://doi.org/10.1109/4235.585892
Respalda: variante ACS, actualización local y parámetro q_0.

[3] M. Dorigo y T. Stützle. Ant Colony Optimization. MIT Press, 2004. https://mitpress.mit.edu/9780262042192/ant-colony-optimization/
Respalda: visión general de ACO, estigmergia, MMAS, valores de parámetros recomendados para el TSP y combinación con búsqueda local.

[4] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza y S. Dizdarevic. «Genetic algorithms for the travelling salesman problem: A review of representations and operators». Artificial Intelligence Review, 13(2):129–170, 1999. https://doi.org/10.1023/A:1006529012972
Respalda: representaciones de rutas (camino, adyacencia, ordinal) y su efecto en la búsqueda.

[5] C. A. Coello Coello. «Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art». Computer Methods in Applied Mechanics and En-

gineering, 191(11–12):1245–1287, 2002. https://doi.org/10.1016/S0045-7825(01)00323-1
Respalda: clasificación de técnicas para manejar restricciones (penalización, reparación, operadores que preservan factibilidad).

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. Introduction to Algorithms, 3.ª ed., MIT Press, 2009, sección 22.1. ISBN 978-0-262-03384-8.
Respalda: definiciones de grafo y representaciones por matriz y por listas de adyacencia, con sus costos.

[7] R. M. Karp. «Reducibility among combinatorial problems». En Complexity of Computer Computations, pp. 85–103, Plenum Press, 1972. https://doi.org/10.1007/978-1-4684-2001-2_9
Respalda: el ciclo hamiltoniano es NP-completo.

[8] M. Haythorpe. «FHCP Challenge Set: The first set of structurally difficult instances of the Hamiltonian cycle problem». Bulletin of the ICA, 83:98–107, 2018. https://arxiv.org/abs/1902.10352
Respalda: 1001 instancias hamiltonianas cuya dificultad proviene de la estructura, no solo del tamaño.

[9] Flinders Hamiltonian Cycle Project. «FHCP Challenge Set» (sitio oficial). https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/fhcp-challenge-set/
Respalda: rango de 66 a 9528 vértices, formato HCP y TOUR, resultado del concurso (985 de 1001 grafos resueltos por el equipo ganador).

[10] L. Pósa. «Hamiltonian circuits in random graphs». Discrete Mathematics, 14(4):359–364, 1976. https://doi.org/10.1016/0012-365X(76)90068-6
Respalda: técnica de rotación para extender caminos hacia ciclos hamiltonianos.

[11] J. C. Bean. «Genetic algorithms and random keys for sequencing and optimization». ORSA Journal on Computing, 6(2):154–160, 1994. https://doi.org/10.1287/ijoc.6.2.154
Respalda: representación por llaves aleatorias.

[12] I. A. Wagner y A. M. Bruckstein. «Hamiltonian(t) – An ant-inspired heuristic for recognizing Hamiltonian graphs». Proc. Congress on Evolutionary Computation (CEC 99), 1999. https://doi.org/10.1109/CEC.1999.782656
Respalda: antecedente de hormigas aplicadas específicamente al ciclo hamiltoniano.

[13] T. Stützle y H. H. Hoos. «MAX–MIN Ant System». Future Generation Computer Systems, 16(8):889–914, 2000. https://doi.org/10.1016/S0167-739X(00)00043-1
Respalda: depósito solo de la mejor hormiga y límites de feromona.

[14] Flinders Hamiltonian Cycle Project, "Flinders Hamiltonian Cycle Project", s. f., https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/. Define el problema hamiltoniano y distingue su objetivo del TSP. El equipo del proyecto, "FHCP Challenge Set", s. f., https://sites.flinders.edu.au/flinders-hamiltonian-cycle-project/fhcp-challenge-set/, describe el conjunto FHCP y su formato. Se citan como fuente institucional de contexto, no como evidencia experimental de este ejemplo.

[15] Michael Haythorpe, "FHCP Challenge Set: The First Set of Structurally Difficult Instances of the Hamiltonian Cycle Problem", 2018, https://arxiv.org/abs/1902.10352. Respalda la identidad y finalidad del conjunto FHCP. La fecha de publicación del artículo de revista es 2018; la versión arXiv enlazada es posterior.

[16] David L. Poole y Alan K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3.ª ed., Cambridge University Press, 2023, sección 3.6, https://www.artint.info/3e/html/ArtInt3e.Ch3.S6.html. Respalda la explicación de búsqueda informada y de función heurística como guía estimativa.

[17] John H. Holland, Adaptation in Natural and Artificial Systems, 1975; reimpresión MIT Press, 1992, https://doi.org/10.7551/MITPRESS/1090.001.0001. Fundamento histórico del algoritmo genético y de la adaptación mediante recombinación y selección.

[18] Xin-She Yang, "Metaheuristic Optimization", Scholarpedia, 2011, https://www.scholarpedia.org/article/Metaheuristic_

Optimization. Respalda el esquema general de GA: codificación, fitness, población, selección, cruce, mutación y reemplazo.

[19] David E. Goldberg y Robert Lingle Jr., "Alleles, Loci, and the Traveling Salesman Problem", Proceedings of the First International Conference on Genetic Algorithms, 1985, https://gpbib.pmacs.upenn.edu/icga/icga1985-ocr.pdf. Respalda el uso de permutaciones y cruces especializados, como PMX, para recorridos.

[20] Derviş Karaboğa, "An Idea Based on Honey Bee Swarm for Numerical Optimization", informe técnico TR06, Universidad de Erciyes, 2005, enlace al informe. Fuente primaria de las fases de ABC, roles de abejas, fuentes, selección y parámetro limit.

[21] Derviş Karaboğa, "Artificial Bee Colony Algorithm", Scholarpedia 5(3):6915, 2010, enlace al artículo. Expone la versión de ABC con selección probabilística, mejora voraz, abandono y controles principales.

[22] Indadul Khan y Manas Kumar Maiti, "A Swap Sequence Based Artificial Bee Colony Algorithm for Traveling Salesman Problem", Swarm and Evolutionary Computation 44, 428–438, 2019, https://doi.org/10.1016/j.swevo.2018.05.006. Respalda que las variantes discretas para recorridos requieren operadores de permutación; no se atribuye a este artículo la adaptación específica propuesta aquí.

[23] Richard M. Karp, "Reducibility Among Combinatorial Problems", en Miller y Thatcher (eds.), Complexity of Computer Computations, Plenum, 85–103, 1972, https://people.irisa.fr/Nicolas.Markey/biblio/ccc1972-Kar. Marco clásico de NP-completitud. Trabajo antiguo y vigente; se verificó la referencia, y queda pendiente confirmar el pasaje específico en el texto.

[24] Michael Haythorpe, "FHCP Challenge Set: The First Set of Structurally Difficult Instances of the Hamiltonian Cycle Problem", Bulletin of the ICA 83, 98–107, 2018, https://arxiv.org/abs/1902.10352. Fuente primaria sobre la composición y dificultad del conjunto, y sobre la adaptación de algoritmos del TSP.

[25] Flinders Hamiltonian Cycle Project, "FHCP Challenge Set", consultado en noviembre de 2026, https://sites.flinders.edu.au/

flinders-hamiltonian-cycle-project/fhcp-challenge-set/. Página oficial: tamaños, existencia de ciclo en todas las instancias y resultados del concurso.

[26] Joeri Sleegers y Daan van den Berg, "Looking for the Hardest Hamiltonian Cycle Problem Instances", ECTA 2020, 40–48, 2020, https://doi.org/10.5220/0010066900400048. Respalda la relación entre densidad y facilidad. Artículo de congreso.

[27] Thomas Bartz-Beielstein et al., "Benchmarking in Optimization: Best Practice and Open Issues", arXiv:2007.03488v2, 2020, https://arxiv.org/abs/2007.03488. Revisión amplia de 17 autores sobre esfuerzo, éxito, fallos, PAR-k, resumen de ejecuciones y semillas. Es un preprint sin arbitraje confirmado.

[28] Nikolaus Hansen, Anne Auger, Dimo Brockhoff, Dejan Tušar y Tea Tušar, "COCO: Performance Assessment", arXiv:1605.03560, 2016, https://arxiv.org/abs/1605.03560. Respalda las evaluaciones como medida, el objetivo fijo, el tratamiento de fallos y el aRT. Su contexto es la optimización continua, de modo que aplicarlo aquí es una transferencia.

[29] A. E. Eiben y J. E. Smith, Introduction to Evolutionary Computing, 2.ª ed., Springer, 2015, https://doi.org/10.1007/978-3-662-44874-8. Respalda eficiencia, efectividad y la advertencia sobre generaciones. Solo se revisaron diapositivas del capítulo; el libro queda pendiente.

[30] Lawrence D. Brown, T. Tony Cai y Anirban DasGupta, "Interval Estimation for a Binomial Proportion", Statistical Science 16(2), 101–133, 2001, https://doi.org/10.1214/ss/1009213286. Respalda la incertidumbre de una tasa de éxito. Se revisó el resumen.

[31] Peter Christen, David J. Hand y Nishadi Kirielle, "A Review of the F-Measure: Its History, Properties, Criticism, and Alternatives", ACM Computing Surveys 56(3), artículo 73, 2023, https://doi.org/10.1145/3606367. Respalda definiciones y críticas de precisión, recall y F1. Trata la clasificación; no se le atribuye la aplicación al ciclo hamiltoniano hecha aquí.

[32] Rainer Storn y Kenneth Price, “Differential Evolution: A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces”, Journal of Global Optimization 11, 341–359, 1997, copia consultada. Fuente primaria del algoritmo, la notación y los parámetros. El DOI queda pendiente de confirmar.

[33] Swagatam Das, Sankha Subhra Mullick y P. N. Suganthan, “Recent Advances in Differential Evolution: An Updated Survey”, Swarm and Evolutionary Computation 27, 1–30, 2016, https://doi.org/10.1016/j.swevo.2016.01.004. Respalda el desarrollo de DE y la adaptación de parámetros. Se revisó el resumen.

[34] Karin Zielinski, Petra Weitkemper, Rainer Laur y Karl-Dirk Kammeyer, “Examination of Stopping Criteria for Differential Evolution based on a Power Allocation Problem”, 10th International Conference on Optimization of Electrical and Electronic Equipment, 2006, https://www.ant.uni-bremen.de/sixcms/media.php/102/4605/ICO3E_2006_zielinski.pdf. Respalda la clasificación de criterios de paro. Un congreso y un solo problema.

[35] James C. Bean, “Genetic Algorithms and Random Keys for Sequencing and Optimization”, ORSA Journal on Computing 6(2), 154–160, 1994, https://doi.org/10.1287/ijoc.6.2.154. Origen de las claves aleatorias, propuestas para algoritmos genéticos. Se revisó el resumen.

[36] Pavel Krömer, Vojtěch Uher y Václav Snášel, “Novel Random Key Encoding Schemes for the Differential Evolution of Permutation Problems”, IEEE Transactions on Evolutionary Computation 26(1), 43–57, 2022, https://doi.org/10.1109/TEVC.2021.3087802. Respalda el uso de claves aleatorias en DE y en el TSP. Se revisó el resumen.

[37] Valentino Santucci, Marco Baioletti y Alfredo Milani, “Algebraic Differential Evolution Algorithm for the Permutation Flowshop Scheduling Problem with Total Flowtime Criterion”, IEEE Transactions on Evolutionary Computation 20(5), 682–694, 2016, https://doi.org/10.1109/TEVC.2015.2507785. Respalda la crítica a las claves aleatorias y la DE algebraica, en otro problema.

[38] Godfrey C. Onwubolu y Donald Davendra (eds.), Differential Evolution: A Handbook for Global Permutation-Based Combinatorial Optimiza-

tion, Springer, 2009, ISBN 978-3-540-92150-9, https://www.wolfram.com/books/profile.cgi?id=7359. Respalda la existencia de varios enfoques. Solo se revisaron la ficha y el índice.

[39] Fatih Tasgetiren, Angela Chen, Gunes Gencyilmaz y Said Gattoufi, "Smallest Position Value Approach", capítulo 5 de [38], 2009, https://www.academia.edu/3714507/Smallest_Position_Value_Approach. Respalda el uso de SPV con DE en un TSP generalizado. Se revisó un extracto; el detalle de la regla queda pendiente.

[40] Leighton, T. y Rubinfeld, R. (2006). Mathematics for Computer Science: Notes for Recitation 6. MIT. Definiciones de grafos, recorridos y longitud. https://web.mit.edu/neboat/Public/6.042/rec6.pdf.

[41] Diestel, R. (2025). Graph Theory, 6.ª ed. Springer, capítulo 10. Definiciones hamiltonianas; referencia conservada de la investigación de SA. https://www.math.uni-hamburg.de/home/diestel/books/graph.theory/preview/Ch10.pdf.

[42] Universidad de Waterloo. The Traveling Salesman Problem. Definición del TSP. https://www.math.uwaterloo.ca/tsp/problem/index.html.

[43] Kirkpatrick, S., Gelatt, C. D. y Vecchi, M. P. (1983). Optimization by Simulated Annealing. Science 220(4598), 671–680. Fundamento de SA. https://doi.org/10.1126/science.220.4598.671.

[44] Hajek, B. (1988). Cooling Schedules for Optimal Annealing. Mathematics of Operations Research 13(2), 311–329. Condiciones de convergencia. https://doi.org/10.1287/moor.13.2.311.

[45] Komosinski, M. Optimization: Simulated annealing. Poznan University of Technology. Material docente citado por el documento original para el pseudocódigo y el enfriamiento. https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/SA-en.pdf.

[46] Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. y Teller, E. (1953). Equation of State Calculations by Fast Computing Machines. Journal of Chemical Physics 21(6), 1087–1092. Antecedente de la aceptación probabilística. https://doi.org/10.1063/1.1699114.

[47] Kennedy, J. y Eberhart, R. (1995). Particle swarm optimization. Proceedings of ICNN'95, vol. 4, 1942–1948. Fuente original de PSO. https://doi.org/10.1109/ICNN.1995.488968.

[48] Shi, Y. y Eberhart, R. (1998). A modified particle swarm optimizer. IEEE International Conference on Evolutionary Computation, 69–73. Variante con peso de inercia. https://doi.org/10.1109/ICEC.1998.699146.